



University of Milano-Bicocca

Department of Informatics, Systems and Communication

Master's Degree Program in Data Science

Multimodal Fusion Strategies in Remote Sensing: Deep Learning for Detecting Marine Pollutants and Sea Surface Features

Supervisor: Prof. Paolo Napoletano

Co-supervisor: Dr. Mirko Paolo Barbato

Master's degree thesis by:

Luca Porcelli

N° 853189

Academic Year 2023-2024

Abstract

Marine pollution is one of the most important environmental challenges of our time, with devastating consequences for ocean ecosystems, biodiversity and human health. Among the main causes are plastic spills, oil spills, chemical and agricultural debris. These factors not only menace marine life, but also compromise the quality of water and food resources that billions of people depend on. The consequences of marine pollution are deep and difficult to manage. Plastic, for example, fragments into microplastics that are carried by marine life to humans. Oil spills damage coastal ecosystems, destroying habitats and causing the death of many species. Given the urgency of this crisis, it is crucial to have tools to effectively monitor and manage these risks. The use of technologies such as automated satellite image analysis can revolutionize the way we identify and manage marine pollution. Using these techniques, pollutants can be detected over vast ocean areas in real time, thereby optimising mitigation and intervention efforts. In this work, however, we do not simply distinguish between polluted and unpolluted areas, but propose a multi-class approach to identify and categorise various aspects of marine pollution. We focus in particular on the data provided by the MADOS dataset, which focuses primarily on marine debris and oil spills, but also takes into account other features present in the marine environment. The dataset used consists of high-resolution multispectral images from the Sentinel-2 (S2) satellite, including 174 scenes collected between 2015 and 2022. These scenes cover a wide geographical distribution, with approximately 1.5 million pixels annotated under varying meteorological conditions. Our main contribution is to re-examine and re-annotate the MADOS ground truth data. Although the existing annotations are of good quality, we have decided to enhance the work by adding denser and more detailed annotations. Subsequently, we decided to use a deep learning model, called **TICINO**, based on multimodal fusion techniques, which we believe is more effective than traditional early fusion. As we will show, this approach allows us to obtain significantly better results than the basic methods. Finally, we will compare our model with the benchmark model, **MariNeXt**, used by the authors of MADOS, and analyse our multimodal fusion approaches by comparing them with the results obtained by **MariNeXt**.

Contents

Abstract	2
1 Introduction	6
2 State of the art	12
2.1 Marine Pollutant Datasets	12
2.1.1 MARIDA	12
2.1.2 MADOS	14
2.2 Data processing	17
2.2.1 Sentinel-2 - Multispectral Data	17
2.3 Multimodal Semantic Segmentation	18
2.3.1 Deep Learning for Semantic Segmentation	19
2.3.2 MariNeXt	22
2.3.3 Ticino	28
2.3.4 Multimodal Fusion Strategies	30
3 Methodology	32
3.1 Problems with MADOS dataset	32
3.1.1 Annotations	32
3.1.2 Upsampling technique	39
3.1.3 Mask-Based Cropping Strategies	41
3.1.4 Class imbalance strategy	44
3.2 Methods	45
3.2.1 Ticino	45
3.2.2 Multimodal Fusion Strategies	46
4 Results	50
4.1 MariNeXt vs Ticino	50
4.2 Multimodal Fusion results	53
4.3 Prediciton with the best model	56
5 Conclusions	60
References	62
Acknowledgements	70

Chapter 1

Introduction

Every year, large quantities of pollutants are discharged into the oceans worldwide. These include waste, plastic, chemicals, sewage, oil, and more. The issue of marine pollution is becoming increasingly relevant, with estimates indicating that 80% of this pollution originates from land-based sources. Considering that the oceans cover about 70% of the Earth's surface, it is clear that this is a global problem that requires growing attention. Marine pollution can manifest in various forms, which can differ greatly from one another. However, in general terms, we can divide it into two main categories of sources:

- **Non-point sources:** These sources cannot be traced back to a single origin but arise from multiple sources distributed across the territory, such as vehicles, live-stock farms, and industries. These pollutants are often accumulated and transported into waterways by rain, eventually reaching the sea. In this case, it is not a single element causing the pollution but the combination of numerous small sources.
- **Point sources:** These originate from a specific area and include events like oil spills or chemical discharges. As highlighted by NOAA, pollution from point sources can have a potentially more significant environmental impact compared to non-point sources, although such events are generally less frequent.

The marine environment is increasingly threatened by the presence of debris, with a growing contribution of plastic pollution, which represents one of the main environmental concerns (Geyer et al.). It is estimated that in 2010 alone, between 4.8 and 12.7 million tons of plastic were released into global oceans from land-based sources. This flow of plastic is expected to increase significantly, with predictions indicating an order-of-magnitude increase within the next decade (Jambeck et al.). Although plastic fragments over time into smaller particles, known as "microplastics" ($0.1\ \mu m - 5\ mm$), most of it will persist in the environment for geological periods (Andrady et al.). The removal of this debris is technically possible, but the process is extremely complex, expensive, and inefficient.

Another significant source of marine pollution is oil spills, which can have severe ecological consequences. The rapid detection of oil spills on the ocean surface is significant for implementing an effective emergency response. Although the detection of these slicks through Remote Sensing (RS) techniques is well established under many operational conditions, estimating the thickness of oil layers remains a technical challenge (Garcia-Pineda et al.). However, conducting extensive measurements is not always practical, safe, or feasible, making the analysis of RS data essential for assessments. Among the most commonly used RS techniques are optical, thermal, and radar sensors mounted on aircraft and satellites (Leifer et al.). These tools provide indispensable support for monitoring

and managing oil spills, although there are still significant limitations in the ability to obtain a comprehensive and accurate assessment of situations in open waters.

In this context, RS emerges as one of the fundamental resources for obtaining a detailed understanding of the territory and its specific characteristics. By using sensors on drones, aircraft, or satellites, RS allows for the collection of images and data related to the Earth's surface, atmosphere, and oceans, thereby facilitating the monitoring and analysis of the environment and its dynamics over time. To provide a visual representation, in Figure 1.1, a satellite in Earth orbit uses sensors to capture detailed information about the Earth's surface. Delving deeper into the concept of RS, we can identify its key elements:

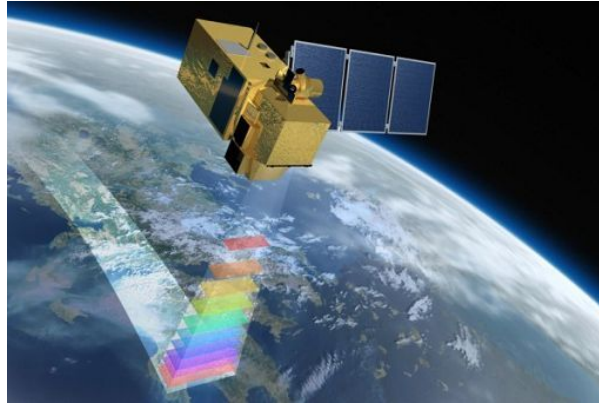


Figure 1.1: *Representation of a satellite in orbit around the Earth, using sensors to collect multispectral data from the Earth's surface. The colored light beams symbolize the collection of information through various spectral bands, essential for environmental analysis and monitoring (ESA / Astrium GmbH).*

- **Remote:** Refers to an operation conducted at a distance, without direct or physical contact with the observed object or phenomenon, which may be located at a short or long distance.
- **Sensing:** Refers to the process of gathering information, data, or measurements, such as images, temperature, or pressure.

When talking about RS, the mind immediately turns to satellite technology. Specifically, RS involves the use of sensors on board satellites orbiting the Earth, capturing images of the Earth's surface from great heights, a typical example of RS. However, the concept of RS is not limited to satellite technology but also includes other techniques such as sonograms, medical imaging, and X-rays, all methodologies that allow for obtaining information about objects or phenomena without direct contact.

A precise definition of RS describes it as the process of measuring or acquiring data related to a property of an object or phenomenon through a device that does not come into physical contact with the observed object. The term "Remote Sensing" was coined in the 1950s by Evelyn Pruitt of the United States Office of Naval Research to describe the science and art of identifying, observing, and measuring an object from a distance. This technology includes numerous satellites, such as Landsat-8, Sentinel-2, Sentinel-3 (Donlon et al.), ASI PRISMA (Candela et al.), and many others, which have transformed our ability to observe and analyze the planet from a global perspective (Salcedo-Sanz et

al.). The analysis of data collected by sensors on satellites or aircraft provides valuable information about the Earth’s land, water, and atmospheric systems.

Thanks to the ease of access to new data, RS has increasingly taken on a central role in sectors such as agriculture (Sishodia et al.), resource exploration (Frick and Tervooren), environmental monitoring (Barbato et al., K. Kikaki et al.), urban planning (Wellmann et al.), and disaster management (Van Westen). For example, in Figure 1.2, we can see six images obtained from the Sentinel-2 satellite for the identification of debris or oil spills in our case. As in other areas of computer vision, the synergy between this



Figure 1.2: *Six images selected from the Mados dataset for the identification of pollutant materials.*

technology, data collection capabilities, and AI methodologies has made it possible to achieve accurate and high-resolution land mapping, accelerating and making the processes of land characterization, segmentation, and mapping more efficient (Ramachandram and Taylor).

Current RS methods often focus on individual marine surface features or a few categories, ignoring competing classes. This approach can frequently lead to false positives. A significant example is the confusion between natural water formations and plastic; however, general debris can also be mistaken for other marine features, such as foam, ships, or, as recent studies have shown, even jellyfish (Qi et al.). To improve the monitoring of marine debris, it is necessary to develop solutions that can accurately identify all marine surface features and reduce false alarms, even under challenging weather and marine conditions. A first step is to consider all materials that may cause confusion in images. The spectral and spatial similarities between some marine categories complicate classification in satellite images. For instance, sea snot can significantly affect debris detection, and discriminating between floating materials in radar data is particularly complex (K. Kikaki et al.). Spectral analyses of optical images, which include competing classes, have shown usefulness, but few studies have applied advanced Machine Learning (ML) methods that use complex schemes to deal with difficult detections, such as small floating objects with complex textures (K. Kikaki et al.). Additionally, the limited availability of public multispectral optical datasets hinders research, complicated by the limitations of current sensors (such as the different band resolutions of Sentinel-2) and the difficulty of identifying and annotating marine surface features, especially in rare events like large plastic patches (K. Kikaki et al.).

Due to the limited focus on the exclusive study of traditional computer vision tasks and thanks to advancements in RS sensors, it is now possible to integrate and utilize new types of data that provide additional information beyond classic RGB data. This allows for the development of multimodal approaches, a topic of growing importance in this research field. The concept of "multimodality", originally linked to mathematics, describes the presence of multiple peaks or modality within a probability distribution (Kress). In simpler terms, it refers to the different channels through which we perceive and express information, such as sight, hearing, speech, and so on (Lloyd and Talja). Think of when we explain something to a friend: we don't just use words, but also the tone of our voice, gestures, and facial expressions. These different elements, which together convey a complete message, are called "modality". They can range from simple signals, like images or sounds, to more complex forms, such as the interpretation of emotions or meanings. Multimodal research focuses on the interaction and integration between these different modality. These can be very similar, like two images of the same scene, or very different, like texts written in different languages (Lloyd and Talja).

In the context of this thesis, these approaches enhance the ability to identify materials on the Earth's surface (Q. He et al.), offering a more detailed and accurate understanding of the scene, especially in complex contexts. In fact, the combination of complementary modality allows for fully leveraging the strengths of each type of information, potentially leading to better results depending on the task and available data. In RS, each modality is often linked to a specific sensor, representing a distinct source of information with unique statistical characteristics (Hong et al.). Beyond conventional RGB modality, RS can benefit from additional data such as LiDAR, multispectral (MS), hyperspectral (HS), panchromatic, and others (Klemas).

This thesis will delve into RS and the multimodal strategy, leveraging satellite images provided by Sentinel-2, a satellite that allows the use of multiple spectral bands. Specifically, the used dataset, 11 bands were used, including the three classic RGB channels, which offer high resolution and cover the visible part of the electromagnetic spectrum. These 11 bands allow for more accurate identification of marine pollution, observing it from different perspectives and providing the model with more learning opportunities to describe each aspect of the image in greater detail. This approach will enable us to demonstrate that multimodality, combined with artificial intelligence, can become one of the most powerful tools available to us.

The images used are multispectral and, as is well known, these images generally have reduced spectral resolution, with a limited number of bands (usually between 5 and 13). In particular, this work focused mainly on the preprocessing phase of the images. The dataset used, known as MADOS, contains images obtained from the Sentinel-2 satellite, focused on detecting debris and oil spills.

A complete annotation of the images was carried out, as the existing one proved to be insufficient and unclear. A study was conducted on the current classes, leading to the decision to add new ones that were considered to be of particular importance, such as land and clouds, while other similar classes were merged, as in the case of different types of algae. Subsequently, given the excessive presence of some classes (such as the marine water), it was decided to randomly select a smaller portion of these large classes, thus creating a mask to apply to the images, avoiding the use of the entire class and preventing an excessively imbalanced dataset.

Finally, a new framework was proposed compared to the one initially suggested by K. Kikaki et al., as it is believed that it could lead to better results, especially through

the use of multimodal fusion methods. In summary, the main contributions of this thesis are:

- Study of multimodality for soil mapping, with particular attention to debris and oil spills.
- Identification of classes.
- Annotation of images with a specific tool, creating the ground truths.
- Creation of masks for some classes.
- Development of a new framework, comparing it with the current one, using multimodal fusion methods for semantic segmentation.

Chapter 2

State of the art

The study of marine pollution and the techniques for its monitoring have gained increasing importance in recent decades. Numerous researchers have analyzed the different sources of pollution and their effects on the marine ecosystem, with a particular focus on the spread of plastic waste and oil spills. Remote sensing techniques have proven essential for monitoring these phenomena, thanks to their ability to cover vast ocean areas and provide real-time data. However, despite significant progress, technological and methodological limitations still persist, complicating the accurate detection and classification of marine debris.

This chapter will examine the existing datasets in detail, describing their construction and content. The analysis will focus on both pollutants (such as oil spills and marine debris) and sea surface features (such as Sargassum macroalgae, marine mucilage, ship-generated waves, and wakes) that can be detected in high-resolution Sentinel-2 (S2) images. The types of data and bands available will also be discussed. Finally, deep learning models will be explored, with a special focus on semantic segmentation, including descriptions of the (MarineXt) benchmark model and the TICINO model, as well as an analysis of multimodal fusion strategies and how they work.

2.1 Marine Pollutant Datasets

Recent studies on oil spill and marine debris detection using Sentinel-2 multispectral data have generally utilized a limited number of images, covering a period of one to two years. Only a few of these studies have used a larger number of scenes or extended data collection beyond six years. Furthermore, the available data are often limited and confined to small geographical areas. Currently, there is no open-access Sentinel-2 dataset specifically for the assessment of oil spill detection methods, while a few open-access datasets have been made available globally for marine debris. Among these, MARIDA is one of the datasets that enables work on marine debris. One of the few large datasets that supports work on both oil spills and marine debris is MADOS, which uses MARIDA along with other collected images and will be utilized in this work.

2.1.1 MARIDA

MARIDA is an open-source dataset that compiles georeferenced polygons and pixel-level annotations on satellite images captured by the Sentinel-2 (S2) sensor. This dataset

was developed to ensure balanced temporal and spatial coverage, using free satellite imagery that focuses on coastal areas around the world. Earth-based reference data on marine debris were gathered through scientist reports and social media information, with particular attention to coastal zones and river mouths. These events were identified in S2 satellite images and verified, when possible, using ultra-high-resolution satellite images. Subsequently, pixels corresponding to marine debris were annotated, along with other visible sea surface features, such as ships, Sargassum algae, sea foam, waves, natural organic materials, different water types, and shallow coastal areas. MARIDA was created to support research on marine debris and the development of operational solutions for its monitoring, including realistic satellite observations that reflect actual conditions, such as clouds or waves, which do not always allow for ideal classification. Annotations were carefully distributed to minimize the risk of errors, considering the complexity of sea surfaces. Annotated polygons do not always represent homogeneous classes or pure pixels; they may include scattered marine debris or floating materials partially obscured by thin clouds.

Between 2015 and 2021, reports on marine debris and plastic pollution in coastal areas and river mouths were collected from several countries. In addition to land-based data, the MARIDA dataset also includes data from previously published studies on satellite monitoring of marine debris (A. Kikaki et al., Biermann et al.) and observations from rivers known to contribute significantly to pollution (Van Emmerik et al., Lebreton et al., Schmidt et al., S. Zhao et al., Jang et al., Cordova and Nurhati).

For the annotations, three image interpretation experts analyzed the data, which included reports, Sentinel-2 satellite images, and spectral indices. These experts annotated marine debris based on real events, taking into account the limitations of the S2 sensor and applying advanced knowledge of the sensor’s spectral behavior (A. Kikaki et al., Topouzelis et al., Biermann et al., Qi et al., C. Hu) and marine debris accumulation models (Cózar et al.). This process required detailed image analysis, with specific pixel-by-pixel annotations. Additionally, floating objects, sea state features, water types, and cloud types were annotated using image interpretation and established spectral models (Dierssen and Garaba, C. Hu et al., Kanjir et al., Y. Liu et al., Kubryakov et al., Dierssen). Wind data were also consulted to avoid confusion between whitecaps and plastic debris (Dierssen and Garaba). Each annotated polygon was classified according to a subject class and assigned a confidence level on a three-grade scale: high, moderate, or low confidence. Following annotation, an expert comparison protocol was adopted to ensure consistency:

- **Marine Debris, Natural Organic Material and Sargassum:** If two annotators agreed on the class, the intersection of the data was maintained, assigning the lower confidence level between the two.
- **Other features:** The merging of annotated data was used. If two different classes were assigned to the same area, the annotation was excluded. If all three experts agreed, the lowest confidence score was used.

In total, MARIDA includes 837377 annotated pixels, of which 3399 pixels represent marine debris. Additionally, 1381 *patch* images are available, each covering an area of $2,4 \times 2,4 km^2$. As shown in Table 2.1, the distribution of samples per class in MARIDA, both at the pixel and *patch* level, is not uniform. As expected, water-related classes are significantly more represented than small floating objects such as plastics. Specifically,

the *Marine Water* class was annotated in 870 *patches*, followed by 373 *patches* of marine debris. This substantial number of annotated images highlights the wide variety of pollutants represented in the dataset. In contrast, classes such as *Mixed Water* and *Natural Organic Material* are the least annotated.

Table 2.1: MARIDA Thematic Classes. Per-class description, number of image patches and corresponding annotated pixels.

Class Name	Description	Patches	Pixels
Marine Debris	Floating plastics, mixed anthropogenic debris	473	3.399
Dense Sargassum	Dense floating Sargassum macroalgae	49	2.797
Sparse Sargassum	Sparse Sargassum	106	2.357
Natural Organic Material	Vegetation & Wood	71	864
Ship	Sailing & Anchored Vessels	182	5.803
Clouds	Clouds	181	117.400
Marine Water	Clear Water & sea state Look-alikes	870	129.159
Sediment-Laden Water	High-Sediment river discharges with brown colour	51	372.937
Foam	Foam recorded at river fronts or coastal wave breaking area	59	1.225
Turbid Water	Turbid waters close to coastal areas	220	157.612
Shallow Water	Coastal waters, including coral reefs and submerged vegetation	64	17.369
Waves	Waves	54	5.827
Cloud Shadows	Cloud Shadows	71	11.728
Wakes	Ship Wakes	106	8.490
Mixed Water	Mixed Water	140	410
Total		1.381	837.377

2.1.2 MADOS

As previously mentioned, one of the few large datasets that allows for work on both oil spills and marine debris is MADOS (an extension of MARIDA), provided by K. Kikaki et al. This dataset includes annotated data from Sentinel-2 (S2), based on 174 scenes and 47 tiles acquired between 2015 and 2022. The dataset contains 1481155 annotated pixels, which have been integrated and organized from various studies, reports, and marine pollution data. MADOS covers 22 countries (Fig. 2.1) and includes 15 different thematic classes (Table 2.2).

Our focus is on two main marine pollutants: *oil spills* and *marine debris*. Regarding oil spills, slicks have been identified from various sources, including ship accidents, oil platforms, oil spills from natural disasters, and natural seepage from the seabed. The goal of our research, as well as that of K. Kikaki et al., is to develop a model that detecting oil spills as a single thematic class, without distinguishing between different types of oil. However, MADOS includes annotations for several oil categories, in line with those established by NOAA (2016), such as: a) Dark, b) Mousse (emulsified), c) Transition Dark, d) Metallic, e) Rainbow, and f) Sheen.

For marine debris, we utilized the open-access dataset *MARIDA* (K. Kikaki et al.), which distinguishes between debris and various sea surface features, while also including debris from additional sources. In line with previous research (Biermann et al., C. Hu et al., A. Kikaki et al., Martínez-Vicente et al.), floating materials that are believed to be spectrally similar to plastic, or that sometimes aggregate with macroplastics on the ocean surface, were also included. Accordingly, *foam*, *Sargassum*, *natural organic*

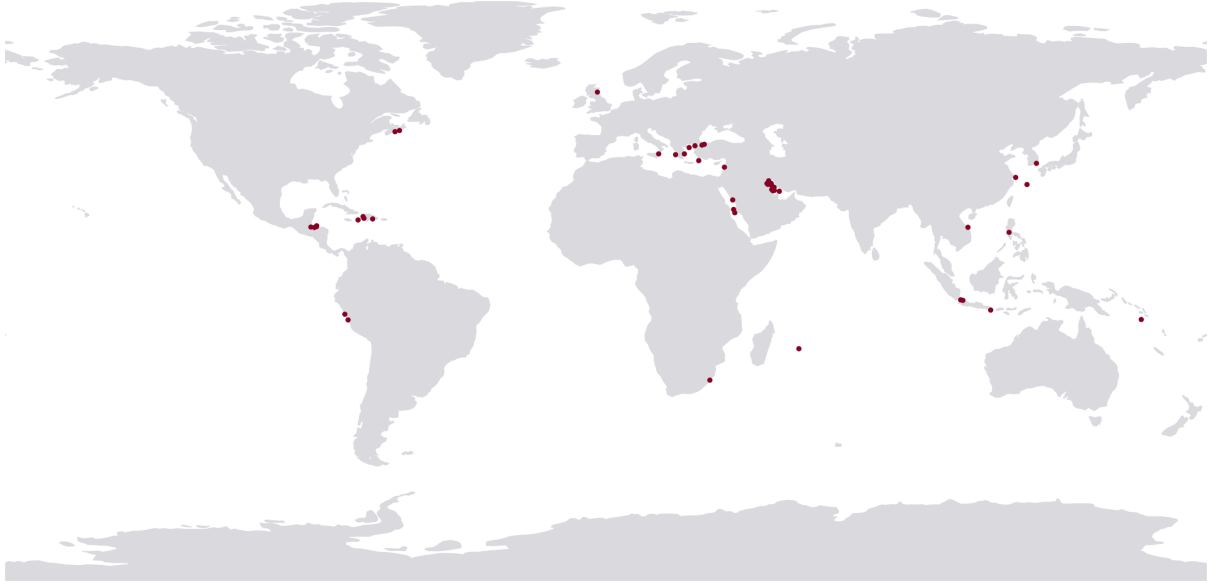


Figure 2.1: *The globally distributed Sentinel-2 tiles from which MADOS data were sampled (K. Kikaki et al.).*

Table 2.2: MADOS Thematic Classes. Per-class description, number of image patches and corresponding annotated pixels.

Class Name	Description	Patches	Pixels
Marine Debris	Floating plastics, mixed anthropogenic debris	472	4.696
Oil spill	Oil types	361	234,568
Dense <i>Sargassum</i>	Dense floating <i>Sargassum</i> macroalgae	77	4.308
Sparse Floating Algae	Sparse <i>Sargassum</i> & other macroalgae	157	4.972
Natural Organic Material	Vegetation & Wood	92	1.454
Ship	Sailing & Anchored Vessels	298	13.222
Marine Water	Clear Water & sea state Look-alikes	1.360	496.702
Sediment-Laden Water	High-Sediment river discharges with brown colour	57	376.061
Foam	Foam recorded at river fronts or coastal wave breaking area	58	1.225
Turbid Water	Turbid waters close to coastal areas	232	170.306
Shallow Water	Coastal waters, including coral reefs and submerged vegetation	140	122.753
Waves & Wakes	Waves & Ship Wakes	224	20.374
Oil Platform	Offshore Oil Platform	348	15.701
Jellyfish	Jellyfish	12	2.541
Sea snot	Sea snot/mucilage	63	12.272
Total		2.803	1.481.155

material (including floating wood), *ships*, and *sea snot* (C. Hu et al.) were noted (Fig. 2.2). Suspected *jellyfish*, as investigated by Qi et al., were also considered to differentiate them from marine debris and sea snot, as previous studies have suggested possible spectral similarities (C. Hu et al., Van der Zande et al.). Additionally, classes that sometimes appear alongside oil spill, such as *oil Platform* and *Sparse Floating Algae*, were noted and integrated. Other water-related classes observed in our study areas, including *Sediment-laden Water*, *Turbid Water*, *Shallow Water*, *Waves*, and *Wakes*, were also annotated. Sparse *Sargassum* was aggregated with other scattered floating algae species, forming a super-class called *Sparse Floating Algae*. This selection of cases is intended to enable

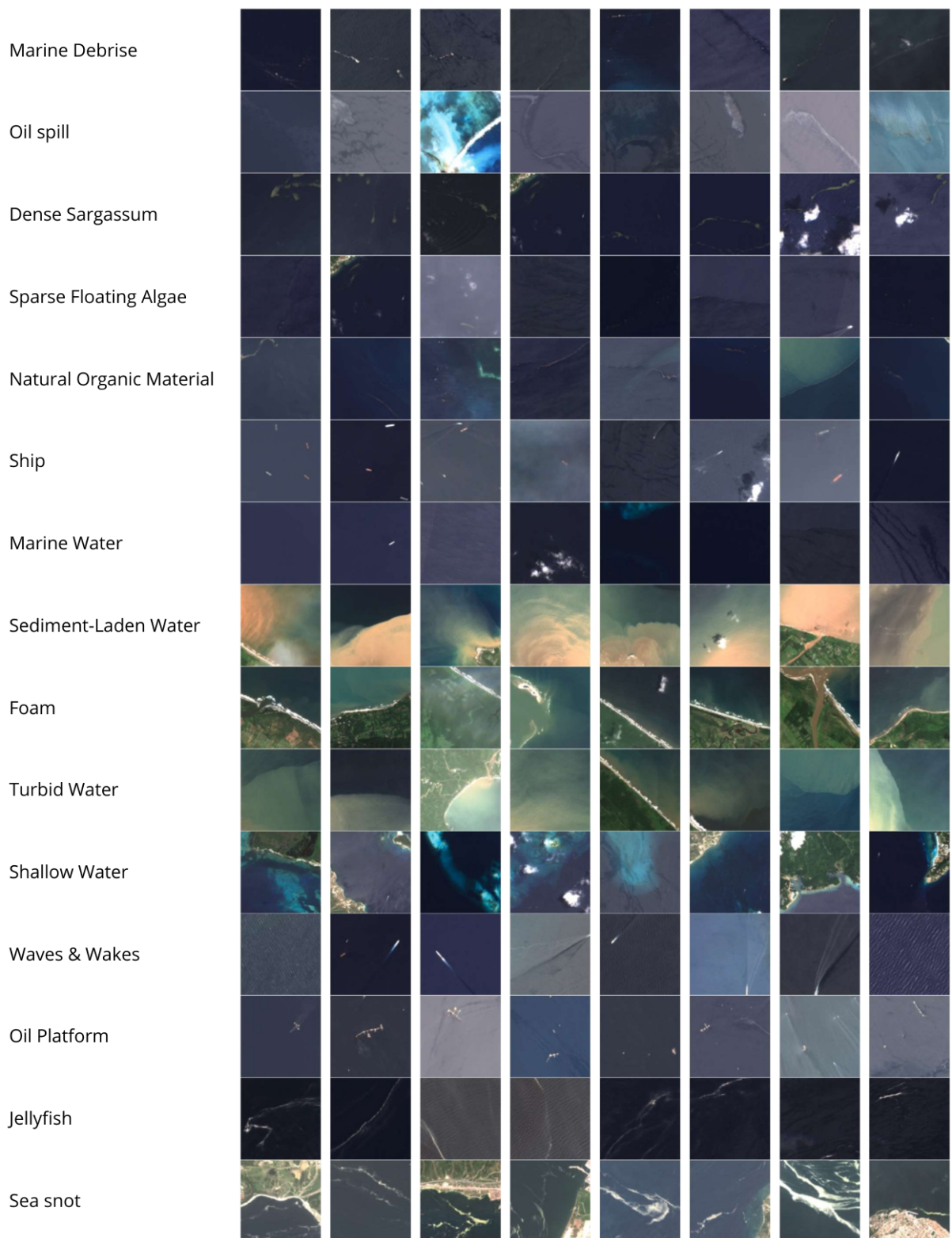


Figure 2.2: *An overview of MADOS provided patches. Marine pollutants and Sea surface features were annotated under various weather and sea state conditions.*

the developed algorithms to distinguish marine pollutants from other floating materials and sea surface features. Clouds and cloud shadows were deliberately excluded from the analysis.

The MADOS dataset contains realistic observations under various meteorological conditions, aimed at supporting practical applications (Fig. 2.2). Overall, MADOS includes 1481155 annotated pixels, of which 4696 pixels represent marine debris and 234568 pixels represent oil spills, corresponding to surface areas of $469600m^2$ and $23456800m^2$, respectively. Additionally, 2803 *patch* images are available, each covering an area of $2,4 \times 2,4km^2$. As shown in Table 2.2, the distribution of samples per class in MADOS, both at the pixel and *patch* level, is not uniform. As expected, water-related classes are significantly more represented than small floating objects such as plastics. Specifically, the *Marine Water* class was annotated in 1360 *patches*, followed by 472 *patches* of marine debris and 361 *patches* of oil spill. This substantial number of annotated images highlights the wide variety of pollutants represented. In contrast, classes such as jellyfish and natural organic material are the least annotated. For these classes, only a few reports were collected with sufficient data to be identified in the S2 satellite images.

2.2 Data processing

To develop operational solutions for monitoring the marine environment, satellite sensors with high spatial, spectral, and temporal resolution are essential. Currently, the freely available multispectral data from the Copernicus Sentinel-2 programme provide global coverage of coastal waters with fine spatial resolutions of 10m, 20m, and 60m. Additionally, Sentinel-2 ensures frequent revisits to observed areas, with an interval of about five days, varying according to latitude. This allows for the collection of a large volume of free geospatial data. The satellite detects data across 13 spectral bands, ranging from visible to Short-Wave Infrared (SWIR). For our study, we used multispectral imagery from the Sentinel-2 satellite, collecting Level 1C products (Top of Atmosphere Reflectance) via USGS EarthExplorer. Following the approach of previous studies on the detection of floating materials (Qi et al., S. Sun et al., J. Zhao et al.), we applied the ACOLITE atmospheric processor (Vanhellemont and Ruddick) for the recovery of Rayleigh reflectance.

2.2.1 Sentinel-2 - Multispectral Data

Sentinel-2 is a European space mission consisting of two twin satellites, Sentinel-2A and Sentinel-2B, which capture detailed images of the Earth (Drusch et al.). These satellites orbit at an altitude of around 786km, positioned 180° apart, meaning they are opposite in their orbits. Sentinel-2 is equipped with sensors capable of acquiring images in 13 different spectral bands, allowing observation of various aspects of the Earth’s surface, such as vegetation, water, and soil (Drusch et al., Louis et al.). The images are acquired at different resolutions: some bands have a resolution of 10, 20 or 60 meters (Gascon et al.), and each image covers an area of 290km across (Gascon et al.). These satellites are designed to image the same area of Earth every five days, ensuring frequent data collection (Louis et al.). The information obtained by Sentinel-2 is used in a variety of applications, including land management, agriculture, disaster preparedness, humanitarian relief, and security (Louis et al.). Sentinel-2 is part of the Copernicus programme

and builds on the work of previous missions such as SPOT and Landsat. The satellites, built by European companies, have an expected lifetime of over seven years, with the possibility of extending operations for up to 12 years, thanks to on-board fuel reserves and batteries. The orbit design ensures consistent lighting conditions, minimizing the effect of shadows and improving image quality over time.

The MultiSpectral Instrument (MSI) on board Sentinel-2 captures images of Earth as the satellite moves along its orbit. It uses a technique called push-broom, in which the sensor acquires data line by line, using the motion of the satellite to continuously obtain new images (Drusch et al.). On average, MSI observes the Earth for about 17 minutes during each pass, with the duration extending up to 32 minutes. Light reflected by the Earth and the atmosphere is collected by a three-mirror telescope, which focuses it onto two sets of sensors (Gascon et al.). These sensors measure light at different wavelengths: ten in the visible and near-infrared (VNIR) range and three in the short-wave infrared (SWIR) (Gascon et al.). To cover a wide area of $290km$, the sensors are arranged in two staggered rows, allowing them to capture high-quality images over large regions (Gascon et al.). The separation of different light bands is achieved by filters mounted on the sensors (Gascon et al.). The MSI system also includes a diffuser to calibrate the instrument and ensure the accuracy of measurements. A shutter protects the instrument from direct sunlight and assists with calibration. Due to the specific arrangement of the sensors, each light band observes the Earth at slightly different times, which can cause small shifts between the images of different bands. However, this design allows the MSI to obtain very accurate measurements over large areas of the Earth.

The 13 spectral bands of Sentinel-2 cover a range from the visible (VNIR) to the shortwave infrared (SWIR):

- 4 bands with 10 meter resolution: the three classic RGB bands (blue ($\sim 493nm$), green ($\sim 560nm$) and red ($\sim 665nm$)) and a near infrared band ($\sim 833nm$).
- 6 bands with a resolution of 20 meters: 4 narrow bands in the red-edge spectral domain of the vegetation VNIR (around 704nm, 740nm, 783nm, and 865nm) and 2 wider SWIR bands (around 1610nm and 2190nm), used for applications such as snow/ice/cloud detection or assessing water stress in vegetation.
- 3 bands with a resolution of 60 meters, primarily focused on cloud screening and atmospheric correction (around 443nm for aerosols and around 945nm for water vapor) and cirrus detection (around 1374nm).

Note: For our project, only 11 bands will be used, as the two 60-meter bands for water vapor and cirrus detection are not included.

2.3 Multimodal Semantic Segmentation

Deep learning is a sophisticated branch of artificial intelligence (AI) and machine learning (ML) that has transformed many industries due to its ability to manage and process large amounts of unstructured data, such as images, audio, and text. An example of deep learning in action is image recognition, a technology that enables systems to identify and classify objects within images, such as distinguishing between a cat and a dog. To understand how image recognition works, it is essential to understand the role of artificial neural networks (ANN). These networks are modeled on the human brain and consist of

layers of nodes (neurons) that process information. During the training phase, the neural network is exposed to a large set of labeled images for which the correct classifications are already known. The system learns by associating the features extracted from the images with their labels. This supervised learning process is based on optimization: the neural network minimizes the error between its predictions and the correct labels by adjusting its internal parameters (weights and biases) through a backpropagation algorithm. Once trained, the network can generalize, meaning it can correctly recognize new images it has never seen before, based on the features it learned during training.

Machine Learning vs Deep Learning Deep learning is a subset of machine learning, but it differs in key ways. In traditional machine learning, feature extraction is often manual, requiring data scientists to identify and select the relevant features of the data. In deep learning, however, feature extraction is automated, thanks to the use of deep neural networks. This automation makes deep learning particularly powerful for processing unstructured data. Additionally, deep learning is highly scalable and effective when handling large amounts of data, making it ideal for applications such as speech recognition, machine translation, and natural language processing (NLP). In contrast, traditional machine learning tends to be less effective in contexts requiring the processing of unstructured data at scale.

Deep Learning Applications The applications of deep learning are vast and continue to expand. In healthcare, for instance, deep learning is used to analyze medical images, enabling early diagnosis of diseases such as cancer. Deep learning models can identify anomalies in radiological images with increasing accuracy, assisting doctors in their decision-making. In the automotive sector, deep learning is the foundation of autonomous driving technologies. Autonomous vehicles, like those developed by Google, use neural networks to analyze their surroundings in real-time, recognizing pedestrians, road signs, and other vehicles, and making autonomous decisions. Other applications include advanced robotics, where humanoid robots, such as Sophia, the first robot to obtain citizenship, in Saudi Arabia, use deep learning to interact with humans and learn from their behaviors. Deep learning is also integral to real-time machine translation, continually improving the quality of translations through continuous learning from new data.

2.3.1 Deep Learning for Semantic Segmentation

Semantic segmentation refers to the task of assigning a class label to each low-level component of a multimedia content. In practice, in the image analysis community, this means classifying each pixel in standard 2D images. The goal is to divide a multimedia content into homogeneous areas that represent the same concept or share the same properties. This process can be compared to creating a hierarchy of spatial elements that contribute to forming a taxonomy of the observed visual scene, as suggested by Barghout.

In other words, semantic segmentation is an essential preliminary step for many advanced applications that require precise image understanding. These include autonomous vehicle control, robotics, and human-assistive technologies, such as medical surgery. More specifically, the goal is to classify each individual pixel of an image, without distinguishing between different instances of the same object. Considering a set of target class labels

$\Omega = \{c_0, c_1, \dots, c_l\}$ and an image consisting of n pixels $I = \{p_0, p_1, \dots, p_n\}$, the task of semantic segmentation is to find the function that assigns each pixel p_i a label c_j . However, this process is closely related to other techniques such as global image classification, object instance detection and segmentation (Kaiming et al.), which instead focus on labeling specific portions of the image and differentiating between object instances. There is also a connection with panoptic segmentation (Kirillov et al.), a more recent approach that aims to achieve a complete image segmentation, distinguishing between different object instances. Figure 2.3 shows these different approaches. Thus, semantic segmentation is a complex problem that requires both local feature analysis and context interpretation and spatial reasoning. Several aspects of this task present significant challenges for creating an effective model that generalizes well to new, previously unseen data.

Semantic segmentation can be performed in different ways, depending on the information available about the data and the semantic level that one wants to reach. Initially, the first approaches relied on unsupervised strategies, where, having a few samples with little or no annotations, pixels were grouped using low-level information such as color, brightness or edge detection. However, more recent advances focus on high-semantic analysis, with the goal of classifying concepts such as objects, scenes or actions, leveraging large annotated datasets. This approach, based on supervised learning, uses samples composed of images and their masks that indicate the correct label for each pixel. This strategy is currently the focus of research, supported by specific benchmarks such as the CityScapes dataset (Cordts et al.).



Figure 2.3: *In the image above, we can observe different segmentation. The top right section shows semantic segmentation, identifying classes like people, land, sky, trees, and cars. The bottom left highlights the detection of individual instances. Finally, the bottom right illustrates panoptic segmentation, which combines both full image semantic segmentation and the detection of instances.*

U-Net One of the most well-known models in the field of semantic segmentation is the U-Net. This model has inspired several subsequent developments, including those that will be discussed later in this thesis. The U-Net was introduced during the 2012 EM segmentation challenge, when Ciresan et al. were the first to outperform traditional segmentation methods using convolutional layers. A key element that contributed to their success was the availability of a large amount of annotated data, which is crucial for convolutional neural networks (CNNs) since they can surpass classical machine learning methods when large datasets are available (O’Mahony et al., Hedjazi et al.).

However, obtaining a sufficient amount of annotated data in the biomedical field presents a complex challenge due to privacy concerns, the complexity of annotation, the specialist skills required, and the high cost of biomedical images (Azad et al.). To address this challenge, Ciresan et al. introduced a method that uses an image patching technique. This approach not only increased the number of samples but also helped model the distribution of data across small image patches. Using a moving window, the CNN could learn visual concepts through a localized representation, although this technique involves an increase in computational load relative to the performance gain achieved (Azad et al.).

In 2015, Ronneberger et al. proposed a new architecture based on the FCN framework of Long et al. during the ISBI Cell Tracking Challenge, where they won by a large margin. The model is based on a fully convolutional network with an encoder-decoder design and data augmentation techniques to overcome data sparsity. Furthermore, the model is reasonably fast and outperformed other methods in the competition. The U-

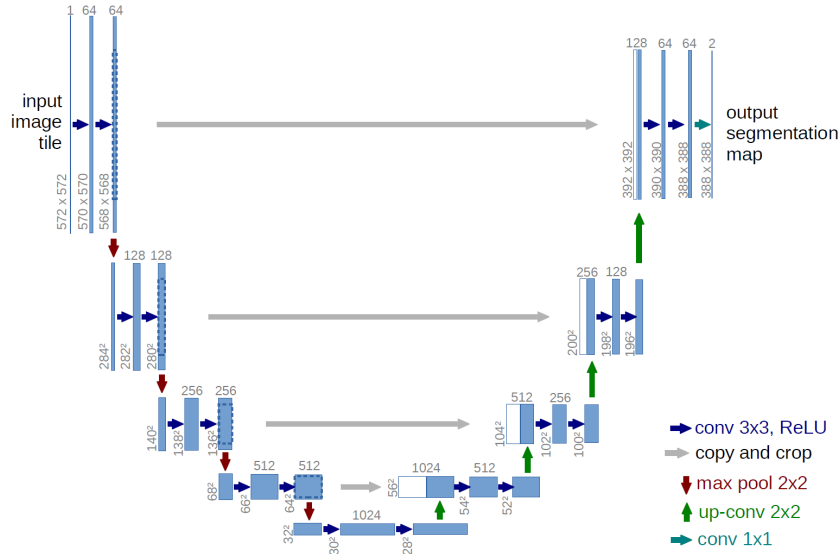


Figure 2.4: *Illustration of the U-Net convolutional network structure. The left side of the U-shape is the encoding stage, with each layer consisting of two 3×3 convolutions with ReLU activation and a 2×2 maximum pooling layer. The right side of the U-shape, consists of the decoding stage and the upsampling process which is realised by a 2×2 deconvolution to reduce the amount of input channels (Ronneberger et al.).*

Net architecture can be divided into two main parts: the encoder and the decoder. The encoder path is responsible for capturing contextual information and is composed of repeated blocks, each with two consecutive 3×3 convolutions followed by a ReLU

activation function and max-pooling layers. Max-pooling is used to gradually increase the receptive field of the network without adding significant computational load. The decoder path, on the other hand, is responsible for gradually upsampling the feature maps until the desired resolution is reached. This path includes a 2×2 transposed convolution layer for upsampling, followed by two consecutive 3×3 convolutions and a ReLU activation function. Between the encoder and decoder paths is the bottleneck, where two consecutive 3×3 convolutions followed by a ReLU activation function are applied. This design allows the network to progressively increase the receptive field and learn richer semantic and contextual representations in the deeper layers.

However, while learning high-level semantic features, the network may lose localization information, which is essential for accurate segmentation. To overcome this problem, Ronneberger et al. introduced skip connections between the encoder and the decoder. These connections concatenate low-level features from the encoder with high-level representations from the decoder, thus improving localization information. Finally, the network uses a 1×1 convolution to map the final representation to the desired number of classes. To reduce the loss of contextual information at the edges of the image, the U-Net adopts a tile overlap strategy.

2.3.2 MariNeXt

In this section, we will examine one of the two models considered, specifically the Benchmark model. Since **MariNeXt** is primarily based on SegNeXt, this section will be divided into two parts. The first part will provide a theoretical overview of some fundamental aspects to better understand the architecture of SegNeXt, followed by a detailed explanation of the model. The second part will describe the modifications made to SegNeXt to create **MariNeXt**.

SegNeXt To understand the SegNeXt model better, as anticipated, we will now look in detail at some aspects that will allow us to better comprehend how it works. One of these is the use of Multi-Scale Networks, particularly applied to their transformer.

Multi-Scale Networks The term *Multi-Scale* in Convolutional Neural Networks (CNN) mainly refers to the feature extraction stage. In other words, the input image is scaled to different resolutions, and a convolution block is applied on each of them to extract features. Multi-scale feature extraction can be achieved through several techniques, including:

- Dilated Convolutions
- Feature Pyramid Network
- Multi-Sized Filter Kernels ($3 \times 3, 5 \times 5, 7 \times 7, 9 \times 9$)

Multi-scale network design is one of the most prominent trends in computer vision (Guo et al.). In segmentation models, multi-scale blocks are present in both the encoder (J. Wang et al., Gao et al., Szegedy et al.) and decoder (H. Zhao et al., Yu and Koltun, L.-C. Chen et al.) parts. GoogleNet (Szegedy et al.), one of the most prominent multi-scale architectures, uses a multi-branch structure to achieve feature extraction at different scales, and it is closely related to our approach (Guo et al.).

As mentioned, a transformer is used in the model. One of the most important aspects to completely understand it is to provide a quick introduction to the mechanisms of attention.

Attention Mechanisms An attention mechanism can be seen as a dynamic and adaptive selection process that allows the network to focus its attention on the most relevant parts of the input (Guo et al.). In other words, it is a method that enables the network to weight features based on their importance, thereby improving the effectiveness of the model. To better understand this concept, we can compare it to how the human brain works. When translating a sentence from one language to another, the brain focuses on specific words or phrases that are most relevant to the task, while ignoring less significant ones. This selection process is what we call "attention" and is a fundamental component of human cognition. Similarly, attention mechanisms in deep learning aim to copy this capacity focusing ability within artificial neural networks. These mechanisms have been introduced to enhance model performance in various tasks, such as image classification (J. Hu et al.), object detection (J. Dai et al.), semantic segmentation (K. Kikaki et al.), image generation (Gregor et al.), super-resolution (T. Dai et al.), and multi-modal tasks (Morelli et al.).

In the context of computer vision, there are six main attention methods, which can be divided into two categories. The first category includes four basic types of attention:

- **Attention on channels:** focuses attention on specific input channels, determining *what* to pay attention to.
- **Spatial attention:** it's about *where* to focus, selecting the relevant areas within the image.
- **Temporal attention:** it deals with the *when* of focusing, identifying key moments in sequential data.
- **Branching of channels:** defines *which* channel to pay attention to among a series of channels.

The second category includes hybrid methods that combine two types of attention:

- **Attention on channel and spatial:** integrates the selection of both channels and relevant spatial positions.
- **Spatial and temporal attention:** combines spatial and temporal dimensions for more precise data selection.

Attention mechanisms have become fundamental in many deep learning applications for their ability to address critical challenges:

- **Long sequence management:** allow models to focus on the most relevant parts, improving the management of complex and long inputs (Vaswani).
- **Contextual understanding:** allow models to assign different weights to words or features based on their context, improving overall understanding (Bahdanau).
- **Improved performance:** they perform better than models without attention mechanisms, especially in tasks such as machine translation, image classification, and speech recognition (Luong).

These mechanisms have revolutionized the field of deep learning, making models more efficient and accurate in a wide range of applications.

At this point, it will be useful to briefly explain how Vision Transformers work. Although a Vision Transformer is not directly used in the SegNeXt, a very similar one is used.

Vision Transformer (ViT) Vision Transformers (Dosovitskiy) are a type of AI model that applies the principles of transformers (originally developed for natural language) to images. To better understand how the Vision Transformer works, we can look at figure 2.5. The image is divided into small squares of fixed size, called patches.

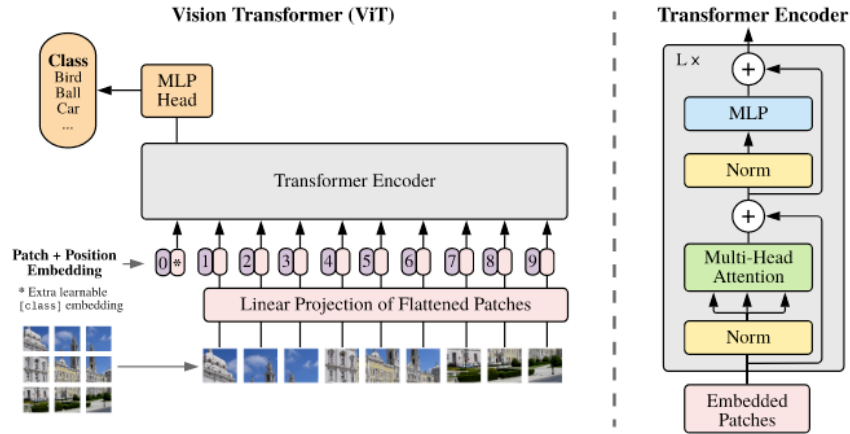


Figure 2.5: *Illustration of the Vision Transformer (ViT) proposed by Dosovitskiy.*

Each patch is then transformed into a vector using a linear projection. These vectors are the inputs to the subsequent layers of the model. Since the Vision Transformer does not automatically understand the spatial relationships between patches, positional information is added to the vectors to provide this information. The heart of the Vision Transformer consists of multiple processing layers, each of which has two main parts:

- **Multi-Head Self-Attention:** this part identifies the relationships between different patches. For each patch, it calculates a weighted combination of the other patches, focusing on the most relevant ones.
- **Feedforward Neural Networks:** after self-attention, the output of each patch passes through a neural network that applies an activation function (such as ReLU) to allow the model to learn complex relationships.

These processes are preceded by level normalization, which makes training more stable and faster, and skip connections, which combine the original input with the output to prevent the vanishing gradient problem.

Finally, another fundamental aspect for understanding the model is the Hamburger method, which will be used in the decoder part. This method, which involves the decomposition of the matrix, gives better results, as Geng et al. also confirmed.

Hamburger Hamburger (Geng et al.) adopts matrix decomposition as the core module for global context, replacing the self-attention mechanism. This choice is motivated by the fact that Hamburger is more computationally efficient, improving its scalability on large amounts of data. Furthermore, it is able to filter noise and isolate the low-rank signal, which represents the most relevant information. Another positive aspect is the greater stability of the results, thanks to the use of One-Step Gradient, which reduces the risk of gradient explosion. Finally, as demonstrated by Geng et al., Hamburger seems to achieve superior performance. Below, a detailed description of the different components of Hamburger:

- **Lower Bread:** This component maps the input Z into a feature space via a matrix of weights, W_i , which represents a linear transformation of dimension $d \times d$. The transformation is computed as $W_i Z$, where i refers to the transformation of a specific step (Lower Bread). This linear transformation prepares the data for further processing by the matrix decomposition module.
- **Ham:** In this stage, matrix decomposition is performed. The transformed input is factorized into sub-matrices to recover a low-rank signal and clean the data from noise. For this purpose, two main models can be applied:
 - Vector Quantization (VQ): A technique that identifies a set of centroids (or prototypes) that best represent the data, minimizing the distance between the original data and the centroids.
 - Non-negative Matrix Factorization (NMF): Here the data matrix is decomposed into two non-negative matrices. NMF tries to find a low-rank approximation of the original data, preserving only the non-negative components.

NOTE: This approach will be used in **MariNeXt**.

The matrix decomposition is represented as $M(X) = DC$, where D (with $D = [d_1, \dots, d_r] \in \mathbb{R}^{d \times r}$) is the dictionary matrix and C (with $C = [c_1, \dots, c_n] \in \mathbb{R}^{r \times n}$) represents the coefficients. The goal is to minimize a loss function $L(X, DC)$, which measures the difference between the original data ($X = [x_1, \dots, x_n] \in \mathbb{R}^{d \times n}$) and the low-rank reconstruction.

- **Upper Bread:** Once the low-rank signal has been recovered, a further linear transformation is applied via a weight matrix W_u to restore the data to its original size, where u refers to the linear transformation of the Upper Bread.

SegNeXt Architecture To describe the SegNeXt architecture clearly, we can divide it into two main components: **Encoder** and **Decoder**, following a similar approach to the U-Net, as explained previously.

- **Encoder:** adopts a structure inspired by the Vision Transformer (ViT) (Dosovitskiy, Xie et al.), with one difference: instead of using the self-attention mechanism, the multi-scale convolutional attention (MSCA) is used. This approach combines the advantages of convolutions with the ability to capture multi-scale contexts, improving the effectiveness of the model on segmentation tasks (Guo et al.). As illustrated in Figure 2.6(a), the MSCA is composed of three main components:
 - Depth-wise convolution: aggregates local information to keep computational cost low.

- Depth-wise multi-branch strip convolutions: Capture multi-scale context using multiple branches, each with a different-sized kernel.
- Convolution 1×1 : models the relationships between different image channels. The output of this convolution is used as an attention map to reweight the input of the MSCA.

Mathematically, the MSCA can be expressed as:

$$\text{Att} = \text{Conv}_{1 \times 1} \left(\sum_{i=0}^3 \text{Scale}_i(\text{DW-Conv}(F)) \right); \quad (1)$$

$$\text{Out} = \text{Att} \odot F; \quad (2)$$

where:

- F represents the input feature.
- Att and Out are the attention map and output respectively.
- $\odot$ indicates the element-by-element multiplication operation between matrices.
- DW-Conv represents depth-wise convolution
- Scale_i denotes the i – th branch of the strip convolutions, with the various kernels of sizes 7, 11 and 21, as shown in Figure 2.5(b).

These large kernels enable the capture of thin features, such as marine debris streaks, without a significant increase in computational cost (Peng et al., Hou et al.). By stacking a sequence of these MSCA blocks, the proposed convolutional encoder, called MSCAN, is constructed. MSCAN adopts a common hierarchical structure, divided into four stages with decreasing spatial resolutions: $H/4 \times W/4$, $H/8 \times W/8$, $H/16 \times W/16$, and $H/32 \times W/32$, where H and W represent the height and width of the input image, respectively. Each stage is composed of a down-sampling block and a stack of MSCA blocks. The down-sampling block uses a convolution with stride 2 and kernel 3×3 , followed by a batch normalization layer.

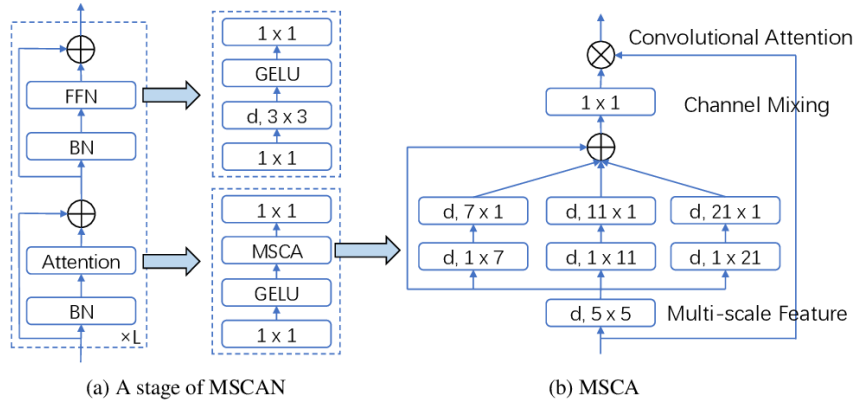


Figure 2.6: *Illustration of the MSCA and MSCAN proposed by Guo et al.*

- **Decoder:** receives features from the last three stages of the encoder, ignoring the low-level information from the first stage because it is not very significant, thus also reducing the computational load. The features are aggregated and then passed through a lightweight Hamburger (Geng et al.) to further model the global context.

MariNeXt Architecture As mentioned, MariNeXt is a variant of SegNeXt with three modifications to achieve better results, which we will explain below.

1st-Stage Higher Resolution Features The first change concerns the SegNeXt architecture, particularly the decision to omit skip connections in the initial phase. Guo et al. opted to exclude low-level information to prioritize the most relevant data, resulting in predictive maps with dimensions $H/8 \times W/8$, which are then upsampled using nearest-neighbor interpolation to match the original image size of $H \times W$. However, in cases where floating materials, such as marine debris, are represented by only a few pixels, high-resolution features become important for accurate modeling (K. Kikaki et al.). This approach allows the model to generate predictive maps with dimensions $H/4 \times W/4$, enhancing performance in marine environmental mapping tasks (K. Kikaki et al.).

Very Simple Copy-Paste (VSCP) The second aspect that has been added is a data augmentation strategy, specifically, Copy-Paste approaches for data augmentation, as they are well-established techniques for object detection and instance segmentation tasks (Dvornik et al., Dwibedi et al., Fang et al.). In particular, this strategy transfers only the annotated pixels to a new image. Recently, Simple Copy-Paste (Ghiasi et al.) proposed placing objects at random positions, showing promising results in instance segmentation. Based on Ghiasi et al., a very simple training augmentation approach, called Very Simple Copy-Paste (VSCP), is proposed, which perfectly fits our task, as illustrated in the example in Figure 2.7. In particular, from the figure, we can see how in the first image there is an annotated oil spill strip (in addition to marine water), while in the second one there is a ship (in addition to marine water). In the third image, the two previous images were superimposed, thus creating a new image that contains both the annotation of the first image and the ship from the second. This approach allows us to obtain a new combined image from the two previous.



Figure 2.7: *The figure shows how the Very Simple Copy-Paste (VSCP) method works in the creation of new images. In the first image, we see marine water with an oil spill, while the second represents a ship with marine water. The third image, combines the ship with the oil spill (K. Kikaki et al.).*

Test-Time Augmentation (TTA) Test-Time Augmentation (TTA) (Kimura, K. Kikaki et al.) is a strategy used to improve the accuracy of an AI model during the testing phase, particularly when the model has to make predictions on new data, such as images it has never seen before. The idea behind TTA is simple: instead of making a single prediction on an image, several transformations, such as rotations, flips, or scalings

are applied to the image, and multiple predictions are made, one for each transformed version.

These multiple predictions are then combined to obtain a more reliable final result. Commonly, this combination is done by averaging the predicted probabilities across all transformed versions, rather than just using majority voting. This averaging process results in a final prediction that takes into account the different variations of the image, leading to a more accurate and stable overall outcome.

2.3.3 Ticino

In this section, we will explore our used model, beginning with an explanation of the ResNet architecture, with a particular focus on ResNet18. We will then analyze its structure in detail, describing in detail how it works, based on ResNet18.

ResNet18 The Residual Network (ResNet) is a deep learning model widely used in computer vision applications, which has revolutionized the field by achieving excellent results, particularly in training very deep neural networks (K. He et al.). One of the main challenges in training deep neural networks is the vanishing gradient problem, where the gradient becomes too small to effectively update the weights in the lower layers of the network. This issue significantly slows down learning in deeper layers and compromises overall network performance (Hochreiter et al.). ResNet addresses this problem by introducing Skip Connections, direct connections between non-consecutive layers that facilitate the flow of the gradient through the network more efficiently (K. He et al.). These connections are implemented through residual blocks, which consist of one or more convolutional layers followed by a Skip Connection. In this way, the gradient can pass directly through these connections without passing all intermediate layers, improving the effectiveness of training (K. He et al.). In addition, ResNet incorporates advanced techniques such as batch normalization and dropout regularization to improve generalization and stability during training (Ioffe, Srivastava et al.).

There are several variants of ResNet architectures, differentiated by the number of stacked residual blocks, which determine their depth. For example, we have ResNet-18, ResNet-34, ResNet-50, and ResNet-101 architectures, where the number indicates the depth of the network. In this context, we refer to ResNet-18, one of the lighter versions.

Figure 2.8 shows the architecture of ResNet-18, allowing for a better understanding of its operation. The architecture begins with a convolutional layer, followed by a pooling

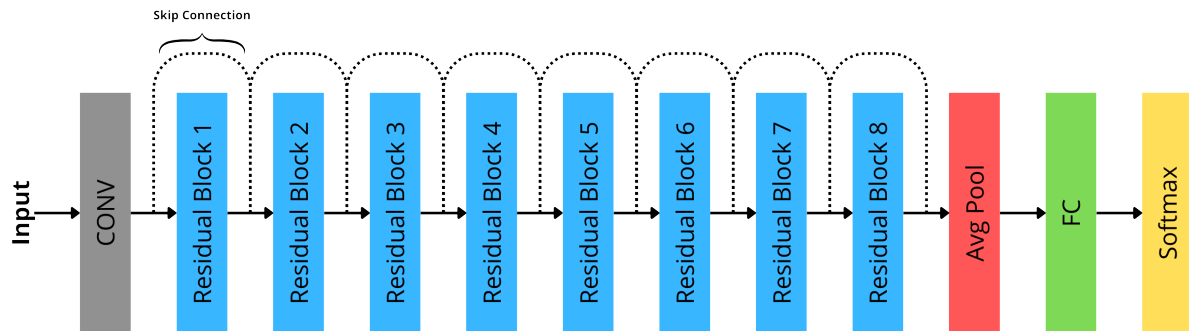


Figure 2.8: *ResNet-18 architecture.*

and normalization operation, which captures the basic features of the image. Next, eight residual blocks are applied, each containing two convolutional layers followed by normalization and ReLU activation. In each block, the skip connections are present, helping to preserve information during propagation. Towards the end, average pooling is applied, reducing the spatial dimensions, followed by a fully connected layer, which brings the total to 18 layers. Finally, a softmax function is used to produce the probabilities for each class.

Ticino Architecture The **TICINO** model has a U-shaped architecture designed for semantic segmentation. In particular, we use the Segmentation Models PyTorch framework¹, which integrates an encoder-decoder network with ResNet18 as the backbone. This design uses the robust visual recognition capabilities of ResNet18, known for its ability to capture complex hierarchical features and improve information extraction. The

Table 2.3: **TICINO** architecture with ResNet18 backbone.

Layer (depth-idx)	Output shape
ResNetEncoder: 1-1	[-1, 3, 256, 256]
Conv2d: 2-1	[-1, 64, 128, 128]
BatchNorm2d: 2-2	[-1, 64, 128, 128]
ReLU: 2-3	[-1, 64, 128, 128]
MaxPool2d: 2-4	[-1, 64, 64, 64]
Sequential: 2-5	[-1, 64, 64, 64]
BasicBlock: 3-1	[-1, 64, 64, 64]
BasicBlock: 3-2	[-1, 64, 64, 64]
Sequential: 2-6	[-1, 128, 32, 32]
BasicBlock: 3-3	[-1, 128, 32, 32]
BasicBlock: 3-4	[-1, 128, 32, 32]
Sequential: 2-7	[-1, 256, 16, 16]
BasicBlock: 3-5	[-1, 256, 16, 16]
BasicBlock: 3-6	[-1, 256, 16, 16]
Sequential: 2-8	[-1, 512, 8, 8]
BasicBlock: 3-7	[-1, 512, 8, 8]
BasicBlock: 3-8	[-1, 512, 8, 8]
UnetDecoder: 1-2	[-1, 16, 256, 256]
Identity: 2-9	[-1, 512, 8, 8]
ModuleList: 2	[]
DecoderBlock: 3-9	[-1, 256, 16, 16]
DecoderBlock: 3-10	[-1, 128, 32, 32]
DecoderBlock: 3-11	[-1, 64, 64, 64]
DecoderBlock: 3-12	[-1, 32, 128, 128]
DecoderBlock: 3-13	[-1, 16, 256, 256]
SegmentationHead: 1-3	[-1, 15, 256, 256]
Conv2d: 2-10	[-1, 15, 256, 256]
Identity: 2-11	[-1, 15, 256, 256]
Activation: 2-12	[-1, 15, 256, 256]
Identity: 3-14	[-1, 15, 256, 256]

process starts with an input of RGB images of size 256×256 pixels. The encoder, based on ResNet18, begins by processing these images through a series of convolutional transformations. The input goes through an initial convolution layer that reduces the spatial dimensions from 256×256 to 128×128 and increases the number of channels to 64. Subsequently, a series of normalization and activation operations (BatchNorm2d and ReLU) stabilize the learning, while a pooling operation further reduces the spatial dimensions

¹https://github.com/qubvel/segmentation_models.pytorch

to 64×64 , maintaining the 64 channels. As the data passes through the various stages of ResNet18, the network applies a series of residual blocks that preserve the information through skip connections. These blocks progressively reduce the spatial dimension while increasing the number of channels, going from 64×64 to 32×32 , then to 16×16 , and finally reaching 8×8 with 512 channels.

The model’s decoder restores the original spatial dimensions of the images, with the DecoderBlock progressively expanding the spatial dimensions: from 16×16 to 32×32 , then to 64×64 , 128×128 , and finally to 256×256 , while the number of channels gradually decreases to 16. This process allows the model to generate a final output of size 256×256 with 15 channels, corresponding to the expected segmentation classes. Table 2.3 shows the architecture.

2.3.4 Multimodal Fusion Strategies

Multimodal techniques involve the study of computer algorithms that learn and improve their performance by integrating data from different modalities. The main objective is to leverage artificial intelligence models to process and find relationships between various types of data (modalities), such as images, video, audio, and text. This approach allows for a deeper understanding from multiple perspectives. For instance, the tone of voice can vary from person to person, and analyzing multiple voices provides a more comprehensive view of the context.

One of the key approaches in this field is multimodal fusion (Barbato et al., Morelli et al., X. Jiang et al.), which has proven successful. Particularly in semantic segmentation, various methods for fusing different modes have been explored, varying based on the dataset used, the model chosen, and the type of fusion applied. The goal is to maximize the benefits of each data type.

In the context of RS, multimodal fusion techniques can be divided into two main categories: heterogeneous fusion and homogeneous fusion (J. Li et al.). Heterogeneous fusion involves integrating modalities with different meanings, such as hyperspectral images. In contrast, homogeneous fusion refers to the combination of data of the same type.

Since convolutional neural networks are used in this field, these fusions can be divided into three: Early fusion, Middle fusion and Late fusion (Y. Li et al.).

- **Early fusion:** concatenation takes place as a first step, directly combining the different data sources as they are initially acquired.
- **Middle fusion:** the different types of data are concatenated in the middle of the architecture, after the extraction of certain preliminary features.
- **Late fusion:** fusion is applied after extracting both low-level and high-level features, thus towards the end of the encoding phase (encoder).

Chapter 3

Methodology

We have reached the most significant chapter of this thesis, the one in which we will express our point of view and present the work carried out, highlighting the choices and thoughts that guided our decisions in specific circumstances. We firmly believe that for this type of research, it is essential to start from accurate and well-structured data. This approach not only ensures the reliability and validity of the results but also helps to minimize distortions and the risk of false positives, a point that we will further demonstrate in the following discussion.

We have decided to divide this chapter into two main sections. In the first part, we will focus on the problems initially encountered in data management, with particular attention to the difficulties related to annotations. In fact, as anticipated, we believe that limited or poorly annotations can generate misleading and difficult-to-interpret results. Subsequently, we will examine some approaches in the context of pre-processing, describing in detail the steps taken to improve the quality of the work and ensure greater theoretical accuracy.

The second part of the chapter will be dedicated to the proposed methods, with a particular focus on the *TICINO* model, which will also be analyzed in some of its variants. We will explore multimodal fusion techniques, testing different approaches and introducing modifications to further optimize the results, trying to reach increasingly higher levels of accuracy.

3.1 Problems with MADOS dataset

3.1.1 Annotations

Annotating images involves assigning labels to each image in a dataset, a critical step in training machine learning (AI) models. By annotating an image, we specify what it contains, such as *dog*, *car*, or *tree*. These labels are then used by the AI to learn to recognize the same objects in new images. After the manual annotation of images is completed, the AI model uses these labels to learn. The ultimate goal is for the model to replicate these annotations on its own, without human intervention. Accurate annotations are essential because any errors in the labels will be replicated by the model, compromising its accuracy. Therefore, high-quality annotation is critical for training neural networks, which form the basis of AI in computer vision.

Images can be annotated manually by humans or by using automated tools. These

automated tools, often based on pre-trained algorithms, can efficiently recognize and annotate objects in images. They are particularly useful for complex tasks, such as creating *segmentation masks*, which would be time-consuming if done by hand. In these cases, automatic annotation can serve as a starting point, which can then be refined manually. Manual annotation can also be facilitated by tools that help identify and mark key points in images, making the process easier and faster. Image annotations are essential for creating the dataset from which AI models learn. The way we label images directly impacts how the AI behaves once trained: if the annotations are inaccurate, the AI will make incorrect predictions.

Annotated data is especially necessary when tackling a new and specific problem with AI. For common tasks, such as image classification or object recognition, there are already pre-trained models that can be adapted to the specific case, using less data thanks to *Transfer Learning*. However, if we need to train a completely new model, a large amount of annotated data is required, divided into training, validation, and testing sets. Creating these sets takes considerable time and resources.

To start annotating images, two things are needed: an effective annotation tool (in our case, the open-source CVAT platform¹) and data of sufficient quality. It is important to consider:

- The type of data (images, videos, etc.)
- The type of annotation needed (labeling objects, creating masks, etc.)
- The format in which the annotations will be saved

Annotations can be used for different tasks, such as classification, where an entire image is assigned to a class, or for object detection and recognition, where annotations take the form of bounding boxes and class names. The extreme coordinates of the boxes and the class ID are set as the GT. Another type of annotation is for image segmentation, specifically semantic segmentation, which is the focus of this case study. Image segmentation divides an image into different parts, each corresponding to a specific class or label. This process goes beyond object detection by aiming to precisely delineate the contours and surface of each object in the image, providing a detailed representation. Annotations for image segmentation are achieved through masks, which are images of the same size as the original and indicate exactly where each object is located. For example, in an image containing a car and a tree, each pixel in the mask is labeled with an ID representing the car, the tree, or the background. Areas that do not belong to any object of interest are marked with a value of zero. The accuracy of these annotations is crucial for the success of segmentation algorithms.

A specific type of image segmentation is semantic segmentation, where the algorithm divides the image into regions of pixels that belong to common categories. For example, if an algorithm is performing semantic segmentation on a photo containing people, cars, and trees, all pixels that represent a person are grouped under the label *person*, without distinguishing between individuals. Figure 3.1 illustrates this process: a satellite image highlights different features, such as *marine water*, *shallow water*, *land*, *clouds*, and *marine debris*, clearly distinguishing these features with numbers, resulting in a matrix indicating the presence of each feature. In semantic segmentation, no distinction is made

¹<https://www.cvat.ai/>

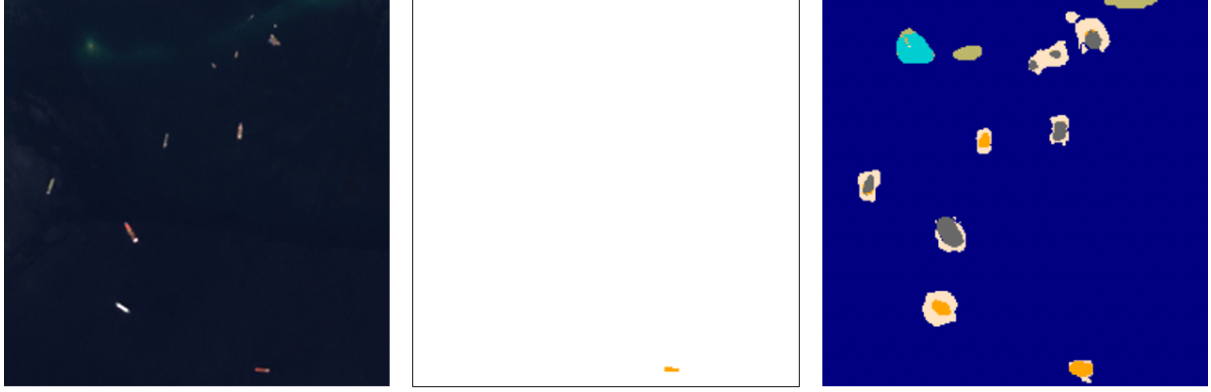


Figure 3.2: *The image shows a comparison between three visual representations: the original RGB image, an obsolete GT where only one ship is highlighted in orange, and a prediction output that incorrectly identifies multiple classes.*

In the third image, while the ship is recognised although not perfectly other unannotated ships were misidentified as oil platforms and wakes or waves were introduced near them. This suggests that the model may have difficulty in accurately predicting marine water.

Another reason for modifying the GTs involves the classes used. In the original dataset of 15 classes, neither land nor clouds were annotated, leading to predictions that do not accurately reflect reality. As shown in Figure 3.3, the model’s prediction aligns well with the annotated parts of the image, resulting in a high score, but the identification of land is completely incorrect, producing an unsatisfactory outcome. Clouds have also

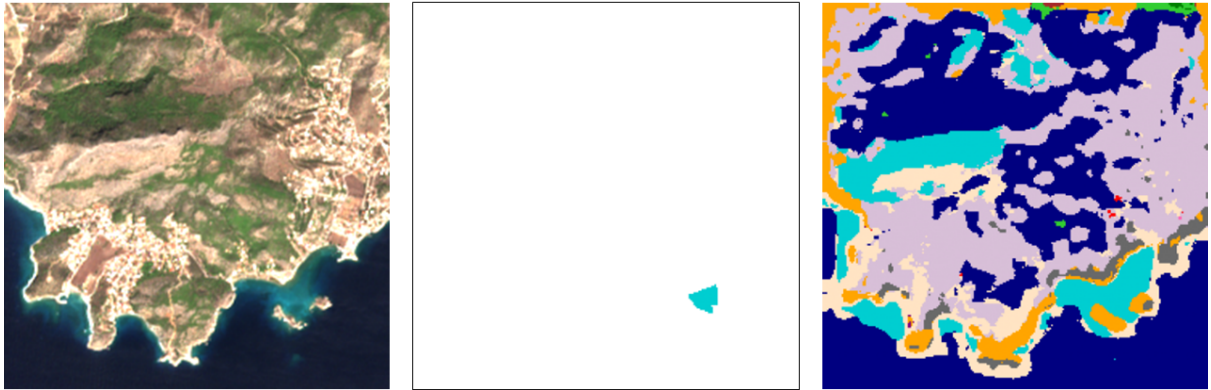


Figure 3.3: *The image presents a comparison between three visual representations: the original RGB image, an obsolete GT where only a section of shallow water is highlighted in light blue, and a prediction output that incorrectly identifies the entire area of the land.*

been included in the annotations for the same reasons. However, we have decided not to annotate shadows. We chose to annotate the features of the area where the shadow is present, ignoring the shadow itself. For example, if a cloud’s shadow fell on the marine water, that area was annotated as marine water, as seen in Figure 3.1, where the cloud’s shadow on the land is still classified as land.

We decided to keep the *wakes* class but removed the *waves* class due to the complexity of their identification and annotation. Annotating every single wave in an image full

of them would be highly challenging, particularly when other features are of interest. Additionally, we merged the *Dense Sargassum* and *Sparse Floating Algae* classes into a single *algae* class, given their similarity. Finally, the *Sea Snot* class was treated as *Foam*, due to their close similarity.

In the last paragraph of the chapter on annotations, it is necessary to address an issue that came up during the analysis: the presence of **burned** images. These overexposed images prevent the correct visualization of the photographed content, likely due to satellite camera malfunctions or other technical problems. The main challenge with these overexposed images is identifying what they represent. Fortunately, the number of burned images is relatively low, and for some, we decided not to proceed with annotation. However, for others, we were able to identify the content through spectral band analysis. In some cases, examining the 11 available bands allowed us to understand what was depicted in the image, enabling accurate annotation. A significant example is shown in Figure 3.4, where a burned RGB image makes it difficult to discern the content due to a large white area. However, the analysis of four higher-quality bands clearly revealed the presence of land, allowing us to correctly annotate that portion of the image as *land*. The new MADOS dataset, together with the new GT annotations, includes a total of

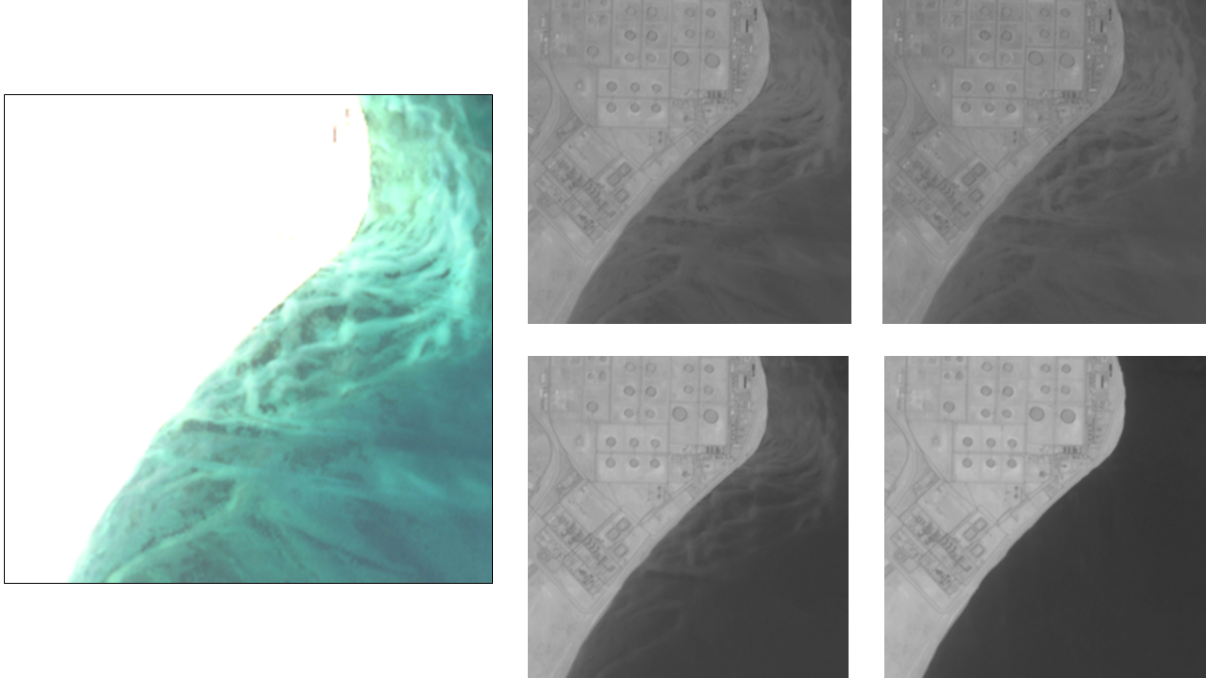


Figure 3.4: *On the left, a colored image displays a RGB image that appears burnt out. On the right, four individual bands (Blue, Green, Red, and Near-Infrared [NIR]) showing the burnt-out areas, indicating the presence of land.*

161063088 annotated pixels, of which 156993 represent marine debris and 1600297 are related to oil spills. In addition, 2803 *patches* of images are available, all annotated, each with a coverage area of $2.4 \text{ times } 2.4 \text{ km}^2$. Table 3.1 highlights how, even in the refining MADOS dataset, the distribution of samples per class, both at pixel and *patch* level, is not uniform, causing an imbalance in the dataset that will have to be corrected during the model training phase.

As expected, even with the new annotations, water-related classes continue to be

Table 3.1: Refining MADOS Thematic Classes. Per-class description, number of image patches and corresponding annotated pixels.

Class Name	Description	Patches	Pixels
Marine Debris	Floating plastics, mixed anthropogenic debris	531	156.993
Land	Land	373	5.599.843
Algae	Dense floating and Sparse Sargassum & other macroalgae	203	83.574
Natural Organic Material	Vegetation & Wood	41	14.693
Ship	Sailing & Anchored Vessels	452	31.620
Oil spill	Oil types & rainbow	360	1,600,297
Marine Water	Clear Water & sea state Look-alikes	2464	128.945.742
Sediment-Laden Water	High-Sediment river discharges with brown colour	92	1.292.378
Foam	Foam recorded at river fronts or coastal wave breaking area	211	306.504
Turbid Water	Turbid waters close to coastal areas	445	16.798.904
Shallow Water	Coastal waters, including coral reefs and submerged vegetation	299	4.256.124
Wakes	Ship Wakes	239	26.808
Oil Platform	Offshore Oil Platform	447	32.727
Jellyfish	Jellyfish	14	16.252
Clouds	Clouds	384	1.900.629
Total		2.803	161.063.088

significantly overrepresented compared to small floating objects like plastics. This imbalance is further accentuated by the new annotations. Specifically, the *Marine Water* class has been annotated in 2464 *patches*, followed by 531 *patches* of *Marine Debris* and 445 *patches* of *Turbid Water*. Classes such as *Jellyfish* and *Natural Organic Material* remain among the least annotated, which, as we will see later, results in lower model performance in accurately recognizing and identifying them.

Finally, Figure 3.5 presents examples of old and new annotations to highlight the differences between the two, compared to the original image. This comparison provides a clear visual representation that helps to understand and justify the work done.

In the first example, we see a large area of land with adjacent shallow water, followed by an expanse of marine water. In the original annotation (GT), these features are almost entirely absent, with only a small area of shallow water detected. In contrast, the new annotation accurately reflects the true content of the image. The second example shows a large area of shallow water, which was poorly represented in the old annotation. In the third example, previously analyzed, we observe numerous ships that were not correctly annotated. The new annotation shows how all the ships, the oil platform, and a small section of shallow water should be represented. Finally, the last example, which is particularly significant and common in this type of annotation, addresses the lack of cloud annotations. This omission impacts predictions since the model was not trained to recognize clouds. However, the most critical issue is the underrepresentation of algae. In the old annotation, the algae were represented by only four pixels, making them almost invisible to the human eye. This is a serious error, as it allows the model to achieve 100% accuracy by simply identifying these four pixels out of a total of 240×240 , without any real understanding of the image content.

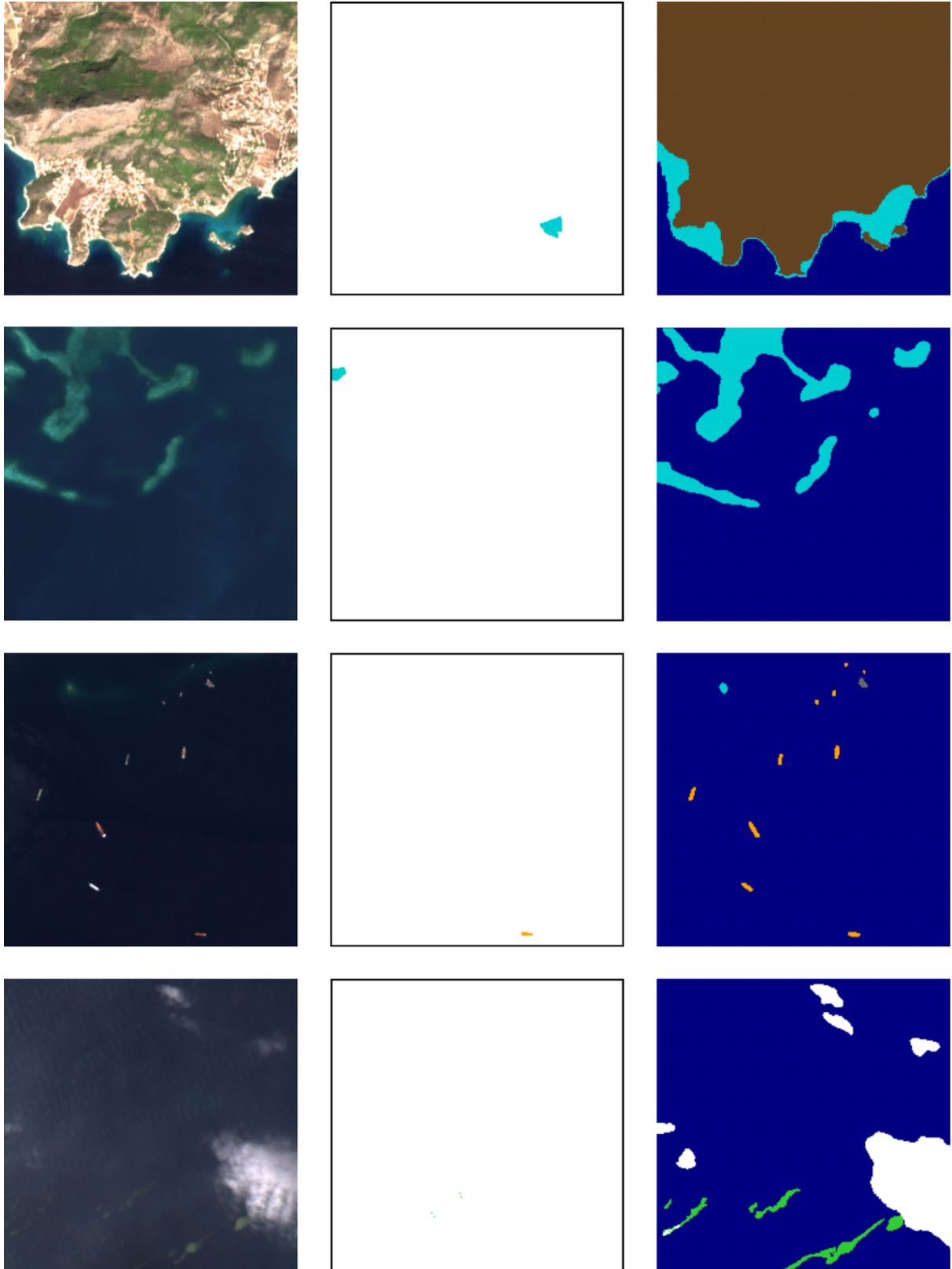


Figure 3.5: The image shows a comparison between three visual representations: the original RGB image, a GT without information, and a new GT that accurately displays all the features of the RGB image.

3.1.2 Upsampling technique

Upsampling aims to increase the resolution of an image, which means increasing the number of rows and columns that compose it (Gonzalez). In other words, it involves expanding the dimensions of the image and filling the new empty spaces created with additional rows and columns. For instance, if we want to triple the size of an image, this would involve adding two rows and two columns for every existing row and column. A simple approach to this is to repeat each row and column of the original image. However, this would only result in a bigger version of the original image, without introducing any new information or improving its quality.

A more advanced and effective method for adding new rows and columns is interpolation, which calculates the intermediate values between existing pixels using sophisticated mathematical techniques. This approach produces a bigger image that not only maintains better visual quality but also features smoother transitions between pixels, avoiding the blocky appearance that simple repetition might cause.

In this thesis, two main strategies for upsampling are used: the Nearest Neighbor method and Bilinear interpolation. The Nearest Neighbor method is simpler and faster but may produce less accurate results. Bilinear interpolation, although more complex, offers higher image quality by better managing the intermediate values between pixels.

Nearest-Neighbor Nearest-neighbor interpolation is a simple method for a bigger image. Suppose we have an image of size 2×2 and we want to double it, resulting in a final image of 4×4 . With this type of interpolation, each pixel in the new image is assigned the value of the closest pixel in the original image. An image is made up of pixels, each with a specific position. Considering the pixels represented by their central value, in an image 2×2 there will be 4 of them, arranged as in the following table:

A	B
C	D

To determine the value of a pixel in the new, bigger image, we first project the grid of the 4×4 image onto the original 2×2 image, yielding this result:

	A	B	
	C	D	

Once the table is enlarged with the new pixel positions in the new image, we proceed to find the closest pixel in the original image, obtaining the following final result:

A	A	B	B
A	A	B	B
C	C	D	D
C	C	D	D

The result is a bigger image where each pixel of the new image takes on the value of the nearest pixel in the original image. Since the nearest-neighbor method does not account for gradual variations between pixels, the final image appears pixelated or blocky, with sharp transitions between colors. A significant advantage of this method is its simplicity

and speed, as it does not require complex calculations. However, the trade-off is in visual quality, as the image is bigger without adding new detail, resulting in a rougher and less detailed appearance. This method is useful in contexts where speed is more important than visual quality.

Bilinear Interpolation Bilinear interpolation is a technique used to estimate an unknown value within a grid, based on a weighted average of the values of the four closest points. In practice, a linear interpolation is performed in two directions: first along the rows and then along the columns. This method is useful for estimating a value between two known points. For example, if we know the values of two points, one equal to 10 and the other equal to 20, we can estimate the intermediate values using a simple proportion. The idea is that the estimated value will be closer to the closest point. To

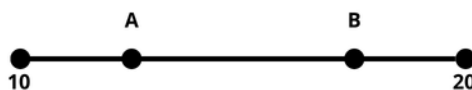


Figure 3.6: *Simple Linear interpolation.*

understand this better, consider Figure 3.6: suppose we have two points, 10 and 20, and we want to estimate the intermediate values between them. Obviously, point A, which is closer to 10, will have a greater weight than point B. The same goes for 20, which will influence point B more. The values of the two points are calculated below:

$$A = 10 \times \frac{3}{4} + 20 \times \frac{1}{4} = 12.5 \quad (3.1)$$

$$B = 10 \times \frac{1}{4} + 20 \times \frac{3}{4} = 17.5 \quad (3.2)$$

When dealing with images, which are two-dimensional, we need to apply this technique in two steps: first along the rows and then along the columns. This process is called bilinear interpolation because we perform the linear interpolation twice, once for each dimension.

Suppose we have an image composed of 4 pixels, which is enlarged to a resolution of 4×4 . The values are placed on the outer edges of the grid, rather than on the inside as in the Nearest Neighbor method:

10	20
30	40

First, we estimate the values along the rows, calculating the points A and B as before (i.e. the two empty boxes between 10 and 20). We then apply linear interpolation to obtain these values. Once we have the intermediate values between 10 and 20, we proceed with another linear interpolation along the columns, to obtain the values between 10 and 30. Repeating this process for all the points in the image, we obtain a smoother final result than simpler techniques such as Nearest Neighbor, as shown below:

10	12.5	17.5	20
15	17.5	22.5	25
25	27.5	32.5	35
30	32.5	37.5	40

However, bilinear interpolation is not ideal for all types of images, as it can sometimes cause blurring.

Nearest-Neighbor vs Bilinear Interpolation Nearest-Neighbor and Bilinear interpolation are two common techniques for scaling images and spatial data, but they differ significantly in terms of quality and complexity. Nearest-Neighbor assigns the output pixel value based on the nearest pixel in the original image, while Bilinear interpolation calculates the output value by considering the grayscale values of the four nearest pixels, using a bilinear function to interpolate between them (Qin et al.).

- **Output quality:** Nearest-Neighbor interpolation selects the closest pixel value, producing an image with sharp edges but lower quality, often characterized by a blocky effect. In contrast, Bilinear interpolation takes into account the four surrounding pixels, producing smoother transitions and a softer, more natural image (Diana Earshia and Sumathi).
- **Computational complexity:** Nearest-Neighbor interpolation is less complex and faster, as it only requires comparing a single distance. Bilinear interpolation, on the other hand, involves calculating a weighted average between four pixels, making the process more complex and time-consuming (Diana Earshia and Sumathi).

Figure 3.7 shows a comparison between the two techniques applied on one of our bands, initially of size 120×120 and then resized to 240×240 . The differences are evident: on the left, the image to which the Nearest-Neighbor interpolation was applied presents a blocky effect, while the image on the right, processed with the Bilinear interpolation, appears smoother but slightly blurry.

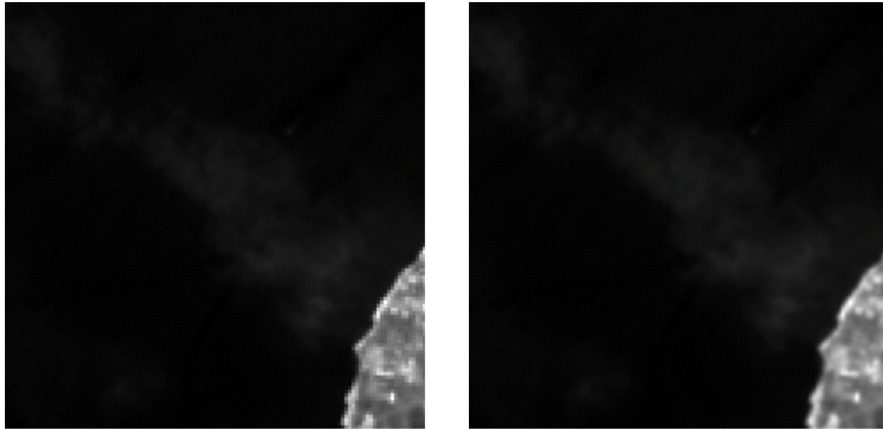


Figure 3.7: *Comparison of nearest neighbor and bilinear interpolation using one of the two SWIR bands capturing 20 meters per pixel.*

3.1.3 Mask-Based Cropping Strategies

As mentioned above, our dataset became significantly unbalanced after re-annotation, particularly for the classes *Marine Water*, *Turbid Water*, and *Land*. These classes are overrepresented in our GTs, and even with a good balancing strategy, the results may still

be suboptimal due to their overabundance. To address this issue, we decided to focus on two strategies specifically applied to these classes. In particular, we concentrated on *Marine Water*, *Turbid Water*, and *Land*. We then developed an image cropping algorithm that extracts only a portion of the large classes to use as a mask on a new GT, devoid of the presence of these classes. The goal was to isolate specific regions of the images to facilitate model training, focusing on areas that might confuse the model during classification.

First Test: Casual Cropping The first attempted approach involved randomly cropping the images to isolate classes with a high concentration of pixels:

- **Identification of Target Pixels:** Identification of the classes of interest (*Land*, *Marine Water*, *Turbid Water*).
- **Quantification of Pixels:** Calculation of the percentage of pixels of the selected classes compared to the total of the image. If this percentage exceeds a pre-set threshold (in our case 10%), the image is cropped around a region containing those pixels.
- **Execution of the Crop:** The crop is performed by selecting a fixed-size window (16×16) around a randomly selected pixel belonging to the class of interest, thus obtaining the mask.
- **Applying the Mask:** The mask obtained is then applied again on the same image, in which the large classes are eliminated.

Figure 3.8 shows an example of the obtained result. In the image on the left, you can see a GT with *Land* represented in brown, adjacent to which *Shallow Water* is visible in light blue. Higher up, there is a strip of *marine debris* in red, while the rest of the image is composed of *Marine Water*. In the image on the right, the final result is shown, where a random part of *Marine Water* has been selected and applied to a new GT without the *Marine Water* class. **Note:** For this example, the procedure has been illustrated only with the *Marine Water* class, but the algorithm is theoretically and practically applicable to *Land* in this image as well. While this method was effective in selecting regions with

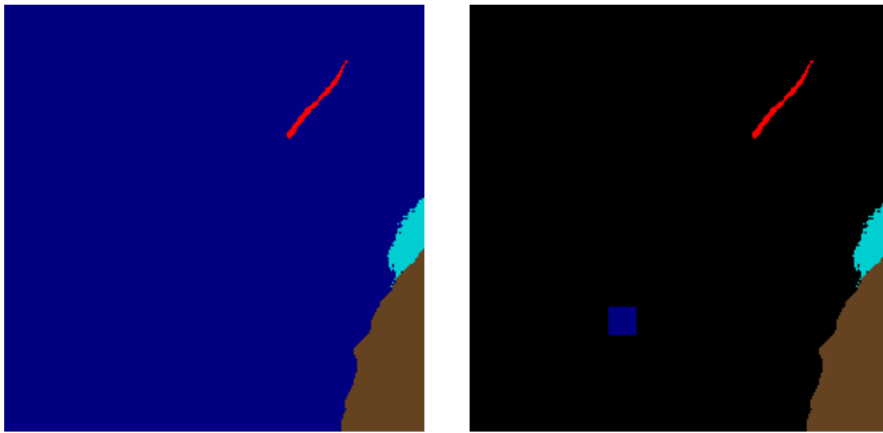


Figure 3.8: *Demonstration of the mask applied for the first test.*

a high concentration of pixels, we realized that it was not optimal for our purpose. The resulting crop did not take into account the surrounding context, such as the presence of pixels from other classes nearby, which could confuse the classification model. In fact, it almost always selected deeper *Marine Water*, often avoiding the water near the coasts or areas with debris. Therefore, we decided to explore a different approach.

Second Test: Boundary Cropping The second approach was developed with the aim of overcoming the limitations of the first method. In this phase, we focused on cropping areas where pixels from large classes were neighboring pixels from other classes, creating more complex situations for the model and promoting more robust training.

- **Pixel Isolation:** Similarly to the first approach, pixels from large classes are identified. However, instead of performing a generic crop, the new method focused on pixels bordering those of other classes.
- **Pixel Dilation of Other Classes:** We use a dilation kernel of size 25×25 pixels to expand the pixels of all classes, creating an edge mask. Dilation is an operation to increase the area of a set of active pixels. By applying the kernel, each pixel of an active class (minor features) affects a surrounding area of 25 pixels, also activating adjacent pixels. This allows us to generate a mask that highlights not only class pixels, but also those that are close to the boundaries with other classes.
- **Pixel Selection of Interest:** Only large pixels that bordered other classes are kept, while the others are discarded. Again, the mask is obtained.
- **Mask Application:** The resulting mask is then reapplied to the same image, eliminating the large classes.

An example of the obtained result is shown in figure 3.9. In particular, the image on

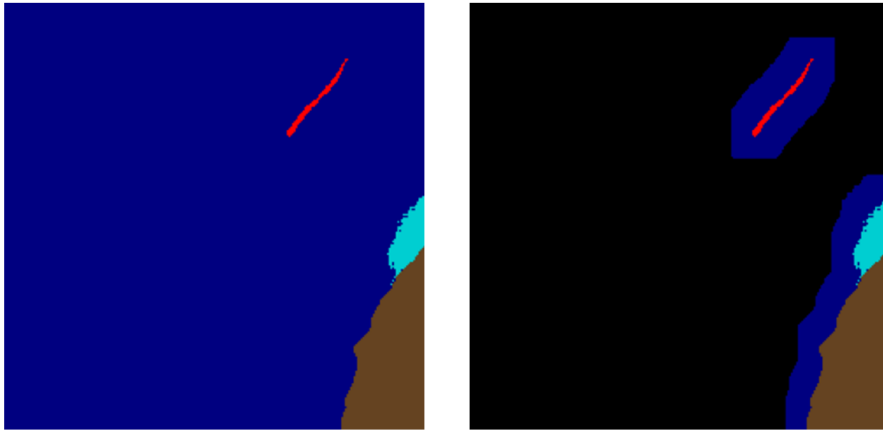


Figure 3.9: *Demonstration of the mask applied for the second test.*

the left represents a GT identical to the one shown in figure 3.8. In the image on the right, instead, we can observe the final result, in which the *Marine Water* has been annotated all in proximity to the other classes, allowing the model to train in more complex circumstances.

This second approach was found to be theoretically more effective, as it allowed us to isolate regions of the image that could lead the model into error, thus making it more robust. The resulting algorithm was then used to generate the new GTs required for training the model.

3.1.4 Class imbalance strategy

Machine learning on unbalanced data, i.e. when the distribution of training data is highly unequal across classes, is a well-known problem (Japkowicz and Stephen, Weiss, H. He and Garcia). In these situations, most algorithms tend to favor the most common classes, i.e. those with more examples in the dataset, leading to poor performance in recognizing less represented classes. However, accurately identifying these minority classes is often crucial, as in the case of rare event discovery (Hospedales et al.). To address this problem, several strategies exist. One of the simplest solutions is to rebalance the training data through techniques such as undersampling the majority classes or oversampling the minority classes, or using a combination of both (Drummond, Holte, et al., Chawla et al., Maciejewski and Stefanowski). Another common strategy is to modify existing learning algorithms to give more weight to the minority classes (Ting, Tang et al., Akbani et al.).

Over the past two decades, many methods have been developed to manage learning with unbalanced data (Krawczyk). However, most of these methods focus on binary and single-label learning problems, often in small-scale contexts and with relatively modest levels of imbalance, such as 1:100 ratios (Dong et al.). These approaches, however, show significant limitations when applied to large-scale data, such as those used in computer vision. Visual data, in fact, are often interpreted through multi-label semantics, as in the case of images of people on the web, which may include multiple attributes related to clothing or facial features. Automatically recognising these properties is very useful (Feris et al.), but it presents a considerable challenge for machine learning, mainly for two reasons:

- **Large-scale imbalance:** Training data for clothing and facial attributes often follow a power-law distribution, where a few classes are very common and many others are rare (Akbani et al.).
- **Thin differences between similar classes:** For example, distinguishing between a *Wool Coat* and a *Cotton Coat* or visually identifying a *Beard* can be very difficult (Dong et al.).

To be able to discriminate these thin classes in a large-scale multi-label context, standard learning algorithms require large amounts of well-balanced training data for all labels (Q. Chen et al., Zhang et al.).

In this case, despite the expected difficulties due to the obvious imbalance in the dataset (although, as mentioned in the previous section, we have reduced the number of pixels for some classes), it was decided to apply weights to the classes. This simple but effective technique helps to manage the imbalance. The idea, as anticipated, is to assign higher weights to minority classes and lower weights to majority classes during the training process. In this way, the model pays more attention to the minority classes, improving the ability to make accurate predictions for them. The steps to implement the class weights are:

- **Understanding data imbalance:** Analyze the distribution of classes in the dataset and identify minority and majority classes.
- **Calculate class weights:** Determine the appropriate weights for each class based on the chosen strategy. In our case, a specific formula was adopted to calculate the weights for each class (K. Kikaki et al.).

$$\frac{1}{\log(1.03 + (\frac{n^{\circ}class}{n^{\circ}total}))} \quad (3.3)$$

Where $n^{\circ}class$ stands for the number of pixels per class and $n^{\circ}total$ stands for the total number of pixels.

- **Implement weights in the model:** The calculated class weights are incorporated into the model’s loss function, ensuring that minority classes have a higher influence during training to address class imbalance.

3.2 Methods

As specified at the beginning of the chapter, we will now illustrate the methods we have developed, with a particular focus on the **TICINO** model and multimodal fusion techniques. We will begin by describing the configuration of the **TICINO** model, which we propose as a competitor to the **MariNeXt** benchmark, presenting the settings for both models, which were configured identically, following the **MariNeXt** setup. Next, we will elaborate on early, middle, and late fusion techniques, applied in different contexts and configurations. Before discussing these aspects, we will briefly refer to the evaluation of the proposed methods, highlighting that the **MADOS** dataset was divided into three subsets: training, validation, and test. The training set is used to train the models, the validation set to monitor the learning process, and the test set to conduct our final evaluation. Additionally, patches cut from each scene are kept within the same set. The splits include 1433 patches for training, 642 for validation, and 728 for testing, with the number of patches and pixels representing approximately 50%, 25%, and 25% of the original **MADOS** dataset, respectively. For the quantitative evaluation of the models’ performance, we used three metrics: the Intersection over Union (IoU), F1 score, and Accuracy.

3.2.1 Ticino

In all experiments conducted, we upsampled the bands 20 m and 60 m resolutions to 10 m using two main approaches: Nearest Neighbour and Bilinear Interpolation, as discussed earlier. For the implementation of the models, we relied on the **MADOS/MARIDA** libraries (K. Kikaki et al., K. Kikaki et al.), **SegNext** (Guo et al.), **timm** (Wightman et al.), and **mmsegmentation** (Contributors). For the initial model (without fusion techniques), we modified the first input layer to handle the 11 S2 bands and the final layer to produce the 15 **MADOS** classes. The model was trained from scratch using the Adam algorithm for 80 epochs, starting with an initial learning rate of 2×10^{-4} , minimizing the Cross-Entropy loss function. After epochs 45 and 65, the learning rate was reduced to 2×10^{-5} and 2×10^{-6} , respectively. The batch size used was 5. (All these settings, as mentioned before, were taken by K. Kikaki et al.).

A significant difference between **MariNeXt** and **TICINO** concerns the resolution of the images. **TICINO** requires images with dimensions divisible by 32, such as 256×256 or 224×224 pixels. However, the images we worked with (those at a resolution of 10 meters per pixel) had dimensions of 240×240 pixels. To address this, we applied a Random Crop, resizing all images to 224×224 pixels. For Data Augmentation techniques in **TICINO** we opted to use Very Simple Copy-Paste (VSCP) during the training phase and Test-Time Augmentation (TTA) during the evaluation phase.

To address the issue of class imbalance, we applied the same weighting scheme used for **MARIDA** and **MADOS** to mitigate the effects of an unbalanced class distribution. During training, we maintained an exponential moving average (EMA) of the model parameters, $\theta' \leftarrow \alpha\theta + (1 - \alpha)\theta$, with α set at 0.999. This technique proved extremely useful in producing more uniform loss curves, as well as more robust and stable models. Specifically, EMA is used to smooth evaluation metrics when training deep learning models. When monitoring values such as loss or accuracy, significant fluctuations can occur between epochs or batches. EMA mitigates these variations, allowing for a clearer observation of the overall trend of the model's performance

3.2.2 Multimodal Fusion Strategies

The choice of using early, middle, or late fusion techniques is complex and must be adapted to the specific context. This process requires a careful evaluation of the trade-offs between feature abstraction and the preservation of spatial information (Barbato et al.). Early fusion combines features with the original data, immediately passing them to the first layers of the network, including the low-level ones. In contrast, middle and late fusion techniques allow for a preliminary features extraction, which are then passed to the model to obtain predictions. These approaches capture both context and global information.

Ealy Fusion As illustrated in Figure 3.10, the pipeline using the early fusion approach consists of concatenating all the modalities into a single representation before being used as input for the model. Specifically, the three modalities we propose are: the

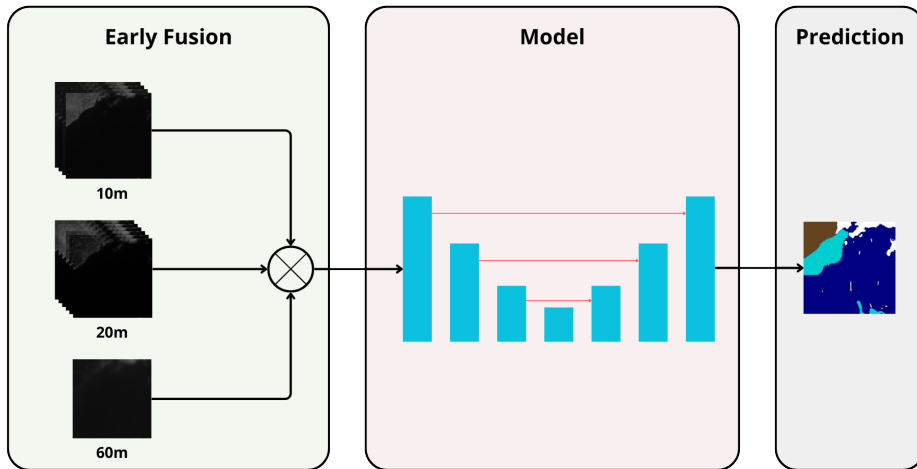


Figure 3.10: This image illustrates the architecture design, showing an early fusion approach where the input data are concatenated immediately before the model.

bands with a spatial resolution of 10 meters per pixel (blue, green, red, NIR), those at 20 meters per pixel (VNIR, SWIR), and finally those at 60 meters per pixel (aerosols).

Middle Fusion Figure 3.11 shows the pipeline using middle fusion. In this approach, the different bands, divided according to image quality, are initially processed separately to extract high-level features from each. Subsequently, these features are concatenated to create the input for the **TICINO** model. The feature extraction phase is based on convolutional layers and ReLU activation functions, which use padding to maintain the same dimensions in terms of width and height as the **TICINO** model. We have developed two middle fusion strategies, one of which is shown in figure 3.11: in this method, for all three band modalities, three convolutional layers are used, each followed by three ReLU layers, with respective dimensions of 128, 64 and 32. This process not only extracts the features, but also increases the number of channels, from the original 11 channels (corresponding to the bands) to 32 channels per modality, for a total of 96 channels that are provided as input to the **TICINO** model. The second approach, on the other hand, is still based on allocating two layers with dimensions of 16 and 32 to all three modalities. In this case, the number of channels increases progressively and constantly, unlike the previous method, where the channels increased suddenly up to 128 and then decreased to 32.

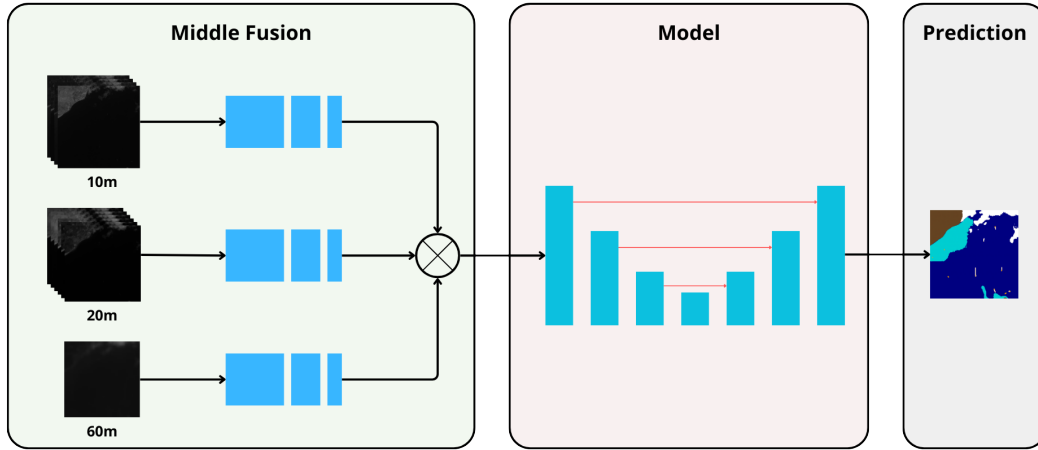


Figure 3.11: This image illustrates the architecture design, showing a middle fusion approach where input data are first processed by three ad hoc CNNs to extract high-level features. These features are then concatenated and passed to the model.

Late Fusion As a last strategy, we have late fusion, illustrated in Figure 3.12. This approach requires the triplication of the encoder for each of the three modalities, with the bands assigned to the respective encoders, as shown in the structure. Subsequently, the resulting features are concatenated and passed to the decoder stage. In addition, skip connections are used, which are also concatenated and then passed on to the respective decoder stages. We experimented with three variants of late fusion:

- **Late Fusion Standard (LFS):** This variant follows the previous description exactly. We tripled the encoder of the **TICINO** model, assigned the corresponding bands, concatenated the outputs and finally passed everything to the decoder, which remains identical to that of the **TICINO** model.

- **Late Fusion variant (LFV):** In contrast to the standard version, this variant also modifies the decoder. In addition to tripling the encoder, we have increased the number of channels in the decoder. Instead of 256, 128, 64, 32 and 16 channels, we used 768, 384, 192, 96 and 48 respectively.
- **Late Fusion without Skip Connections (LFnSC):** In this strategy, based on the standard version of late fusion, we decided to remove the skip connections for the 20 and 60 meters per pixel bands, as we believe that, being of lower quality, they may provide less relevant or even misleading information.

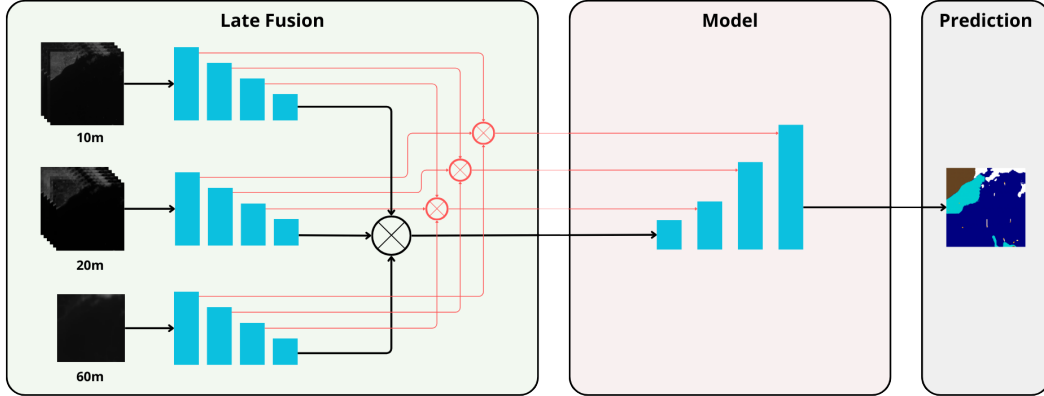


Figure 3.12: This image illustrates the architecture design, showing a late fusion approach where each dataset is passed through its own encoder, with skip connections concatenated at corresponding blocks. The extracted features are then combined and passed to the decoder.

Chapter 4

Results

This chapter will present and discuss the results obtained from our **TICINO** model, compared with the benchmark **MariNeXt** model. For a better understanding, the chapter has been divided into three main sections.

The first section focuses on the one-to-one comparison between **TICINO** and **MariNeXt**, with the aim of determining which model performs better in terms of accuracy, analysing the performance for each class. Special attention has been paid to the classes of marine debris and oil spills, which are essential for correctly evaluating the level of marine pollution. In addition, overall metrics will be presented to provide a general overview of performance. The metrics used, as mentioned earlier, are the IoU, Precision and F1-score. For each metric and model, the highest score will be highlighted in bold to facilitate the interpretation of the results.

The second section, on the other hand, illustrates the results obtained from multimodal fusion using the **TICINO** model. Here we compare the performance of **Early Fusion**, already shown in the comparison with **MariNeXt**, with that of **Middle Fusion** (evaluated using convolutional layers of different sizes (128, 64 and 32)) and the three techniques of **Late Fusion**: the standard version, the variant with decoder modification and the version with removal of the **skip connections**, at band resolutions of 20 and 60 meters per pixel.

The third section presents the RGB images, the newly created GTs, and the predictions obtained from the best model identified in the comparison of the first two sections. This section is aimed at providing a visual representation of the model's performance.

An important note concerns the process of running the models. The models were run five times consecutively to ensure the robustness of the results. This choice was due to the fact that the **MariNeXt** presented stochasticity problems, making it difficult to obtain equivalent performance for each run (despite using a seed). In order to allow a equal comparison (between the two models), aleatoricity was also introduced in the **TICINO** model, by running 5 runs for each model and for each type of fusion proposed. This allowed for more representative and comparable results between the two models.

4.1 MariNeXt vs Ticino

Now we will examine the results obtained by comparing the two models we worked on. Table 4.1, structured on 16 rows, shows the performance of the 15 classes plus a row for the final overall, with all the metrics used. The next four columns present the results for

four types of models:

- **MariNeXt NN:** This benchmark model uses an upsampling approach based on the nearest neighbour method.
- **MariNeXt Bilinear:** This benchmark model uses an upsampling approach based on the bilinear method.
- **Ticino NN:** This is our proposed model, which uses an upsampling approach based on the nearest neighbour.
- **Ticino Bilinear:** This is our model using an upsampling approach based on the bilinear method.

Let us begin by analysing the overall results. From Table 4.1, it is evident that the benchmark model **MariNeXt** definitely overperforms the model proposed by us. In particular, with the nearest neighbour approach, **MariNeXt** shows a significant difference compared to **TICINO**:

- **IoU:** **MariNeXt** is more than 16% higher than **TICINO**.
- **Precision:** **MariNeXt** exceeds **TICINO** by about 11%.
- **F1-score:** The difference is almost 18% in favour of **MariNeXt**.

These data clearly indicate that **MariNeXt** offers a higher overall quality than **TICINO**.

Concerning the use of the bilinear approach versus the nearest neighbour, for the **MariNeXt** model, the bilinear approach shows an improvement of about 3 % in the overall for each metric. For the **MariNeXt** model, the results are very similar between the two approaches: the nearest neighbour shows slightly higher values for IoU and F1-score (44% and 56.8% respectively), while precision is better with the bilinear (60%).

Looking at the performance for each class, it is clear that larger classes tend to score better than smaller ones, with less frequent classes such as Jellyfish and Natural Organic Material showing wider variation. In contrast, classes like Marine Water and Land are more stable. In particular, **MariNeXt** demonstrates greater stability across all classes, particularly in ambiguous classes like Foam and Oil Platform, suggesting a robust handling of data variability. This may be due to **MariNeXt**'s architecture, which appears to generalize better across different environments, while **TICINO** exhibits greater variance, especially in smaller or less distinct classes.

For the classes of particular interest, i.e. marine debris and oil spills, the best model, **MariNeXt**, achieves an IoU of 26.5% for marine debris and 14.6% for oil spills. While these results are better than those from **TICINO**, they are still suboptimal for effectively identifying these features of marine pollution. The complexity and variability of these objects in marine environments likely pose significant challenges for accurate detection and classification by the models.

Class	Metric	MariNeXt NN	MariNeXt Bilinear	Ticino NN	Ticino Bilinear
Marine Debris	IoU	26.5 ±0.6	26.2 ±1.1	10.3 ±3.8	16.5 ±3.7
	Precision	30.9 ±0.8	30.6 ±1.7	10.5 ±3.9	17.1 ±3.7
	F1-score	42.0 ±0.7	41.5 ±1.4	18.5 ±6.3	28.2 ±5.5
Land	IoU	95.1 ±0.0	95.0 ±0.1	82.7 ±6.0	77.8 ±17.0
	Precision	96.0 ±0.1	95.9 ±0.1	83.7 ±6.8	80.2 ±18.3
	F1-score	97.5 ±0.0	97.5 ±0.0	90.4 ±3.6	86.3 ±12.0
Algae	IoU	50.7 ±0.4	51.1 ±1.3	50.7 ±1.2	44.3 ±7.3
	Precision	65.8 ±1.7	62.1 ±3.5	69.2 ±1.9	57.9 ±12.4
	F1-score	67.3 ±0.4	67.7 ±1.2	67.3 ±1.1	61.0 ±7.5
Natural Organic Material	IoU	14.1 ±3.0	14.7 ±1.2	9.3 ±4.1	13.4 ±9.1
	Precision	50.0 ±7.8	68.4 ±7.6	24.6 ±14.1	34.0 ±17.4
	F1-score	24.5 ±4.6	25.5 ±1.8	16.7 ±6.8	22.6 ±13.2
Ship	IoU	22.5 ±1.0	25.5 ±2.2	24.4 ±6.9	26.7 ±1.6
	Precision	49.8 ±2.2	49.0 ±2.7	32.7 ±9.8	39.0 ±2.3
	F1-score	36.7 ±1.3	40.6 ±2.8	38.6 ±9.5	42.2 ±2.0
Oil Spill	IoU	14.6 ±0.4	15.7 ±3.8	2.9 ±0.5	4.5 ±1.5
	Precision	15.2 ±0.5	16.4 ±4.2	2.9 ±0.5	4.6 ±1.5
	F1-score	25.5 ±0.6	26.9 ±6.0	5.5 ±1.0	8.6 ±2.7
Marine Water	IoU	87.3 ±1.6	87.0 ±0.9	45.0 ±4.1	50.8 ±4.8
	Precision	95.2 ±0.4	95.1 ±0.3	98.0 ±0.3	98.0 ±0.2
	F1-score	93.2 ±0.9	93.1 ±0.5	62.0 ±3.8	67.2 ±4.3
Sediment-Laden Water	IoU	60.3 ±0.8	54.9 ±2.3	5.6 ±0.9	6.1 ±2.8
	Precision	75.7 ±1.8	64.6 ±4.1	5.8 ±1.0	6.4 ±3.1
	F1-score	75.2 ±0.6	70.8 ±1.9	10.7 ±1.7	11.3 ±5.1
Foam	IoU	52.3 ±0.6	51.7 ±2.4	38.4 ±11.6	49.4 ±1.0
	Precision	69.4 ±1.1	66.6 ±1.0	45.7 ±15.8	60.6 ±1.9
	F1-score	68.7 ±0.5	68.1 ±0.1	54.4 ±12.7	66.1 ±0.9
Turbid Water	IoU	54.7 ±2.6	51.7 ±2.4	28.3 ±3.3	35.0 ±6.5
	Precision	86.0 ±1.3	83.1 ±0.8	35.3 ±5.6	44.0 ±8.9
	F1-score	70.7 ±2.2	68.1 ±2.0	44.1 ±4.1	51.5 ±7.3
Shallow Water	IoU	38.0 ±6.0	36.2 ±2.6	26.7 ±6.5	20.6 ±2.8
	Precision	42.9 ±7.6	40.5 ±3.2	28.5 ±7.3	21.6 ±3.0
	F1-score	54.8 ±6.6	53.1 ±2.9	41.7 ±8.3	34.1 ±3.9
Wakes	IoU	11.5 ±0.7	12.4 ±2.0	11.7 ±2.0	11.8 ±2.6
	Precision	19.4 ±2.3	21.8 ±4.4	16.9 ±3.2	16.4 ±4.6
	F1-score	20.6 ±1.1	21.9 ±3.2	20.9 ±3.2	21.0 ±4.4
Oil Platform	IoU	35.7 ±0.4	38.1 ±1.1	47.1 ±1.5	47.6 ±2.1
	Precision	38.5 ±0.6	41.5 ±1.4	55.4 ±4.2	54.7 ±2.4
	F1-score	52.6 ±0.4	55.2 ±1.2	64.0 ±1.4	64.5 ±1.9
Jellyfish	IoU	23.2 ±2.7	11.4 ±7.7	7.6 ±4.6	4.7 ±4.0
	Precision	65.7 ±5.9	79.6 ±8.0	31.3 ±22.1	31.3 ±28.1
	F1-score	37.6 ±3.6	19.6 ±12.2	13.8 ±8.1	8.7 ±7.2
Clouds	IoU	73.4 ±0.6	73.7 ±1.1	23.3 ±19.2	36.5 ±20.3
	Precision	82.9 ±0.3	85.0 ±0.8	24.5 ±21.1	39.0 ±22.9
	F1-score	84.7 ±0.4	84.8 ±0.7	34.4 ±21.5	50.5 ±20.6
Overall	IoU	44.0 ±0.6	43.0 ±0.4	27.6 ±2.7	29.7 ±2.3
	Precision	58.9 ±0.9	60.0 ±0.7	37.7 ±4.6	40.3 ±4.1
	F1-score	56.8 ±0.7	55.6 ±0.6	38.9 ±3.1	41.6 ±2.4

Table 4.1: Performance comparison between MariNeXt and TICINO.

4.2 Multimodal Fusion results

Table 4.2 shows all the results of the multimodal fusion models based on **TICINO**. As expected, fusion techniques prove to be more effective than simple early fusion, as they allow for better feature capture and more accurate image interpretation. In general, the approach that distinguishes itself the most is the **Late Fusion** with variant (LFV), which shows a clear improvement: almost 13% increase for IoU, more than 14% for precision and about 13% for F1-score. This allows the model to be close to the results of **MariNeXt**, positioning itself as a competitive alternative to the values previously obtained by the later. A additional strength of late fusion is the better stability of performance between runs, being more consistent than early fusion. However, it does not always achieve the same reliability as **MariNeXt**; for example, for the class *sediment-laden water*, the standard deviation is 16.1, a high value especially when compared to the results obtained by **MariNeXt**. Regarding classes of interest, although LFV always performs better than early fusion, it is still far behind the performance of **MariNeXt**, which continues to outperform.

The **Middle Fusion** performed better than early fusion, but not as strongly as LFV. For this reason, we have decided to terminate further experiments on these classes, as they do not seem to promise significant improvements, contrary to what we observed with late fusion.

Finally, the approach of removing **skip connections** failed, leading to worse results than early fusion. This suggests the importance of maintaining information from images with resolutions of 20 and 60 meters per pixel, because these contain useful data for processing.

In Table 4.3, multimodal fusion techniques are again presented, this time using a **bilinear** approach. In contrast to what was observed in Table 4.1, the bilinear approach seems to improve the overall performance, except for the middle fusion technique which is worse than using the **nearest neighbour**, performing even worse than early fusion. The performance of middle fusion approaches close to that of late fusion without skip connection. However, for other techniques, in particular LFV, there are some small improvements, bringing the model closer and closer to **MariNeXt**. For instance, **MariNeXt** obtains a 44% in the IoU metric, while LFV stands at 41.2%, with a difference of less than 3%. This result encourages us to continue with the optimisation, in the hope that new late fusion techniques will allow us to outperform our benchmark model.

In general, the results follow a similar trend to previous techniques, with slight but consistent improvements. A further observation concerns the standard deviation, which remains significant. For example, for the class *sediment-laden water*, the standard deviation reaches a value of 20 for precision.

All models tested and compared confirm that **MariNeXt** remains the best performing model, initially showing a large gap compared to **TICINO**. However, thanks to the application of multimodal fusion techniques, we were able to significantly reduce this gap, leading **TICINO** to achieve almost comparable results to **MariNeXt** with only a small percentage difference.

Class	Metric	Early Fusion	Middle Fusion (128, 64, 32)	Late Fusion		
				LFS	LFV	LFnSC
Marine Debris	IoU	10.3 $\pm$ 3.8	19.6 $\pm$ 1.1	16.9 $\pm$ 7.8	23.1 $\pm$ 5.3	8.6 $\pm$ 3.1
	Precision	10.5 $\pm$ 3.9	20.4 $\pm$ 1.2	17.5 $\pm$ 8.1	24.2 $\pm$ 5.8	8.8 $\pm$ 3.2
	F1-score	18.5 $\pm$ 6.3	32.8 $\pm$ 1.5	28.1 $\pm$ 12.6	37.2 $\pm$ 7.2	15.8 $\pm$ 5.5
Land	IoU	82.7 $\pm$ 6.0	94.3 $\pm$ 0.6	92.3 $\pm$ 3.5	95.8 $\pm$ 0.1	75.6 $\pm$ 9.3
	Precision	83.7 $\pm$ 6.8	95.5 $\pm$ 0.3	96.1 $\pm$ 0.5	96.6 $\pm$ 0.1	85.3 $\pm$ 11.8
	F1-score	90.4 $\pm$ 3.6	97.1 $\pm$ 0.3	96.0 $\pm$ 2.0	97.9 $\pm$ 0.1	85.8 $\pm$ 6.5
Algae	IoU	50.7 $\pm$ 1.2	44.9 $\pm$ 1.6	51.2 $\pm$ 1.6	53.1 $\pm$ 1.0	40.8 $\pm$ 5.3
	Precision	69.2 $\pm$ 1.9	55.2 $\pm$ 4.9	59.3 $\pm$ 2.4	61.4 $\pm$ 2.1	45.7 $\pm$ 6.9
	F1-score	67.3 $\pm$ 1.1	62.0 $\pm$ 1.5	67.7 $\pm$ 1.3	69.3 $\pm$ 0.9	57.7 $\pm$ 5.6
Natural Organic Material	IoU	9.3 $\pm$ 4.1	1.8 $\pm$ 2.6	22.1 $\pm$ 12.1	46.5 $\pm$ 2.6	7.6 $\pm$ 3.3
	Precision	24.6 $\pm$ 14.1	16.2 $\pm$ 19.0	32.0 $\pm$ 20.8	83.2 $\pm$ 2.8	8.3 $\pm$ 3.7
	F1-score	16.7 $\pm$ 6.8	3.4 $\pm$ 4.8	34.7 $\pm$ 16.2	63.4 $\pm$ 2.4	13.9 $\pm$ 5.6
Ship	IoU	24.4 $\pm$ 6.9	26.7 $\pm$ 1.5	26.6 $\pm$ 4.6	28.2 $\pm$ 3.1	27.0 $\pm$ 2.3
	Precision	32.7 $\pm$ 9.8	37.9 $\pm$ 3.1	33.7 $\pm$ 6.8	41.3 $\pm$ 4.1	41.9 $\pm$ 5.4
	F1-score	38.6 $\pm$ 9.5	42.1 $\pm$ 1.8	41.8 $\pm$ 5.7	43.8 $\pm$ 3.7	42.4 $\pm$ 2.8
Oil Spill	IoU	2.9 $\pm$ 0.5	2.2 $\pm$ 0.6	7.3 $\pm$ 2.5	10.4 $\pm$ 3.5	3.9 $\pm$ 1.1
	Precision	2.9 $\pm$ 0.5	2.2 $\pm$ 0.6	7.4 $\pm$ 2.5	10.6 $\pm$ 3.6	3.9 $\pm$ 1.1
	F1-score	5.5 $\pm$ 1.0	4.3 $\pm$ 1.1	13.5 $\pm$ 4.3	18.7 $\pm$ 5.7	7.4 $\pm$ 2.1
Marine Water	IoU	45.0 $\pm$ 4.1	40.5 $\pm$ 2.4	57.8 $\pm$ 7.8	57.7 $\pm$ 5.2	46.4 $\pm$ 3.1
	Precision	98.0 $\pm$ 0.3	97.8 $\pm$ 0.3	98.3 $\pm$ 0.1	97.8 $\pm$ 0.3	98.1 $\pm$ 0.5
	F1-score	62.0 $\pm$ 3.8	57.6 $\pm$ 2.4	73.0 $\pm$ 6.2	73.0 $\pm$ 4.1	63.3 $\pm$ 2.9
Sediment-Laden Water	IoU	5.6 $\pm$ 0.9	22.4 $\pm$ 4.1	31.7 $\pm$ 11.1	45.2 $\pm$ 9.7	4.8 $\pm$ 1.2
	Precision	5.8 $\pm$ 1.0	24.2 $\pm$ 4.6	37.6 $\pm$ 17.1	59.1 $\pm$ 16.1	4.9 $\pm$ 1.3
	F1-score	10.7 $\pm$ 1.7	36.4 $\pm$ 5.4	47.1 $\pm$ 12.2	61.6 $\pm$ 9.4	9.1 $\pm$ 2.2
Foam	IoU	38.4 $\pm$ 11.6	47.1 $\pm$ 2.7	44.6 $\pm$ 6.2	55.2 $\pm$ 0.6	24.3 $\pm$ 5.3
	Precision	45.7 $\pm$ 15.8	54.6 $\pm$ 4.2	50.4 $\pm$ 7.8	67.2 $\pm$ 1.3	26.3 $\pm$ 6.1
	F1-score	54.4 $\pm$ 12.7	63.9 $\pm$ 2.4	61.4 $\pm$ 6.0	71.2 $\pm$ 0.5	38.8 $\pm$ 7.3
Turbid Water	IoU	28.3 $\pm$ 3.3	32.5 $\pm$ 1.6	37.2 $\pm$ 3.6	39.7 $\pm$ 4.4	36.5 $\pm$ 3.2
	Precision	35.3 $\pm$ 5.6	44.8 $\pm$ 2.9	50.7 $\pm$ 8.7	52.2 $\pm$ 8.8	48.8 $\pm$ 5.4
	F1-score	44.1 $\pm$ 4.1	49.0 $\pm$ 1.8	54.2 $\pm$ 4.0	56.7 $\pm$ 4.5	53.4 $\pm$ 3.5
Shallow Water	IoU	26.7 $\pm$ 6.5	14.1 $\pm$ 3.3	13.0 $\pm$ 2.6	11.0 $\pm$ 2.1	22.0 $\pm$ 3.4
	Precision	28.5 $\pm$ 7.3	14.5 $\pm$ 3.5	13.3 $\pm$ 2.7	11.2 $\pm$ 2.2	22.8 $\pm$ 3.6
	F1-score	41.7 $\pm$ 8.3	24.5 $\pm$ 5.1	22.9 $\pm$ 4.1	19.7 $\pm$ 3.3	35.9 $\pm$ 4.5
Wakes	IoU	11.7 $\pm$ 2.0	10.4 $\pm$ 1.8	6.2 $\pm$ 1.3	5.8 $\pm$ 1.3	10.4 $\pm$ 4.0
	Precision	16.9 $\pm$ 3.2	18.7 $\pm$ 2.0	6.9 $\pm$ 1.6	6.4 $\pm$ 1.6	13.3 $\pm$ 6.2
	F1-score	20.9 $\pm$ 3.2	18.7 $\pm$ 2.9	11.7 $\pm$ 2.3	10.9 $\pm$ 2.3	18.6 $\pm$ 6.6
Oil Platform	IoU	47.1 $\pm$ 1.5	40.7 $\pm$ 2.8	52.4 $\pm$ 2.1	52.9 $\pm$ 2.4	50.4 $\pm$ 2.5
	Precision	55.4 $\pm$ 4.2	50.6 $\pm$ 1.2	58.8 $\pm$ 2.3	58.2 $\pm$ 3.6	55.8 $\pm$ 3.3
	F1-score	64.0 $\pm$ 1.4	57.8 $\pm$ 2.9	68.8 $\pm$ 1.8	69.2 $\pm$ 2.0	66.9 $\pm$ 2.2
Jellyfish	IoU	7.6 $\pm$ 4.6	1.8 $\pm$ 0.9	10.7 $\pm$ 7.1	2.3 $\pm$ 1.0	9.8 $\pm$ 9.3
	Precision	31.3 $\pm$ 22.1	11.5 $\pm$ 6.9	30.4 $\pm$ 18.7	33.9 $\pm$ 6.6	32.3 $\pm$ 18.9
	F1-score	13.8 $\pm$ 8.1	3.5 $\pm$ 1.8	18.7 $\pm$ 11.8	4.4 $\pm$ 1.8	16.5 $\pm$ 14.5
Clouds	IoU	23.3 $\pm$ 19.2	51.6 $\pm$ 16.1	60.1 $\pm$ 15.7	76.2 $\pm$ 1.2	13.6 $\pm$ 5.3
	Precision	24.5 $\pm$ 21.1	58.8 $\pm$ 19.7	63.7 $\pm$ 17.4	82.8 $\pm$ 1.6	13.7 $\pm$ 5.4
	F1-score	34.4 $\pm$ 21.5	66.4 $\pm$ 15.6	73.8 $\pm$ 12.7	86.5 $\pm$ 0.7	23.5 $\pm$ 8.5
Overall	IoU	27.6 $\pm$ 2.7	30.0 $\pm$ 1.3	35.3 $\pm$ 2.5	40.2 $\pm$ 0.5	25.4 $\pm$ 0.9
	Precision	37.7 $\pm$ 4.6	40.2 $\pm$ 1.8	43.7 $\pm$ 3.4	52.4 $\pm$ 1.4	34.0 $\pm$ 1.7
	F1-score	38.9 $\pm$ 3.1	41.3 $\pm$ 1.3	47.6 $\pm$ 2.6	52.2 $\pm$ 0.5	36.6 $\pm$ 1.3

Table 4.2: Performance comparison of different fusion methods with nearest neighbor.

Class	Metric	Early Fusion	Middle Fusion (128, 64, 32)	Late Fusion		
				LFS	LFV	LFnSC
Marine Debris	IoU	16.5 $\pm$ 3.7	21.4 $\pm$ 1.2	17.7 $\pm$ 5.1	25.4 $\pm$2.2	12.0 $\pm$ 4.3
	Precision	17.1 $\pm$ 3.7	22.3 $\pm$ 1.5	18.2 $\pm$ 5.3	26.7 $\pm$2.4	12.3 $\pm$ 4.5
	F1-score	28.2 $\pm$ 5.5	35.2 $\pm$ 1.6	29.7 $\pm$ 7.5	40.4 $\pm$2.8	21.2 $\pm$ 6.9
Land	IoU	77.8 $\pm$ 17.0	92.1 $\pm$ 4.1	89.8 $\pm$ 4.6	95.8 $\pm$0.1	67.8 $\pm$ 10.0
	Precision	80.2 $\pm$ 18.3	95.3 $\pm$ 0.1	95.3 $\pm$ 2.2	96.6 $\pm$0.2	79.2 $\pm$ 13.5
	F1-score	86.3 $\pm$ 12.0	95.8 $\pm$ 2.3	94.6 $\pm$ 2.6	97.9 $\pm$0.1	80.4 $\pm$ 7.5
Algae	IoU	44.3 $\pm$ 7.3	44.7 $\pm$ 0.9	51.2 $\pm$ 1.0	54.0 $\pm$0.5	34.3 $\pm$ 3.1
	Precision	57.9 $\pm$ 12.4	49.4 $\pm$ 1.6	59.3 $\pm$ 1.5	64.0 $\pm$1.8	37.8 $\pm$ 3.9
	F1-score	61.0 $\pm$ 7.5	61.7 $\pm$ 0.9	67.7 $\pm$ 0.9	70.1 $\pm$0.5	51.0 $\pm$ 3.5
Natural Organic Material	IoU	13.4 $\pm$ 9.1	1.8 $\pm$ 1.8	33.6 $\pm$ 4.5	45.9 $\pm$0.9	5.3 $\pm$ 4.3
	Precision	34.0 $\pm$ 17.4	4.5 $\pm$ 5.2	50.1 $\pm$ 8.8	79.4 $\pm$2.8	5.7 $\pm$ 4.8
	F1-score	22.6 $\pm$ 13.2	3.4 $\pm$ 3.4	50.2 $\pm$ 5.2	62.9 $\pm$0.9	9.7 $\pm$ 7.7
Ship	IoU	26.7 $\pm$ 1.6	20.2 $\pm$ 6.4	28.3 $\pm$ 5.6	31.4 $\pm$2.0	24.5 $\pm$ 4.2
	Precision	39.0 $\pm$ 2.3	28.7 $\pm$ 10.5	36.2 $\pm$ 8.3	43.6 $\pm$3.8	36.3 $\pm$ 8.0
	F1-score	42.2 $\pm$ 2.0	33.0 $\pm$ 9.6	43.8 $\pm$ 7.3	47.8 $\pm$2.3	39.1 $\pm$ 5.4
Oil Spill	IoU	4.5 $\pm$ 1.5	2.6 $\pm$ 0.7	3.6 $\pm$ 0.7	7.6 $\pm$1.9	3.8 $\pm$ 1.1
	Precision	4.6 $\pm$ 1.5	2.6 $\pm$ 0.7	3.6 $\pm$ 0.7	7.7 $\pm$1.9	3.8 $\pm$ 1.1
	F1-score	8.6 $\pm$ 2.7	5.0 $\pm$ 1.4	6.9 $\pm$ 1.3	14.0 $\pm$3.3	7.2 $\pm$ 2.1
Marine Water	IoU	50.8 $\pm$ 4.8	40.2 $\pm$ 4.1	46.0 $\pm$ 4.4	54.7 $\pm$5.3	47.7 $\pm$ 2.1
	Precision	98.0 $\pm$0.2	97.9 $\pm$ 0.1	97.8 $\pm$ 0.3	97.7 $\pm$ 0.2	98.2 $\pm$ 0.4
	F1-score	67.2 $\pm$ 4.3	57.2 $\pm$ 4.1	62.9 $\pm$ 4.1	70.6 $\pm$4.4	64.6 $\pm$ 1.9
Sediment-Laden Water	IoU	6.1 $\pm$ 2.8	11.0 $\pm$ 6.8	20.4 $\pm$ 8.7	48.2 $\pm$10.0	4.7 $\pm$ 2.2
	Precision	6.4 $\pm$ 3.1	11.4 $\pm$ 7.1	23.1 $\pm$ 10.3	68.0 $\pm$20.0	4.9 $\pm$ 2.3
	F1-score	11.3 $\pm$ 5.1	19.2 $\pm$ 10.4	32.9 $\pm$ 13.0	64.4 $\pm$9.2	9.0 $\pm$ 4.0
Foam	IoU	49.4 $\pm$ 1.0	46.0 $\pm$ 3.9	44.8 $\pm$ 12.1	54.9 $\pm$1.3	34.2 $\pm$ 4.8
	Precision	60.6 $\pm$ 1.9	54.8 $\pm$ 4.7	51.6 $\pm$ 15.2	67.1 $\pm$3.2	37.7 $\pm$ 5.7
	F1-score	66.1 $\pm$ 0.9	62.9 $\pm$ 3.8	60.9 $\pm$ 12.6	70.8 $\pm$1.1	50.7 $\pm$ 5.5
Turbid Water	IoU	35.0 $\pm$ 6.5	28.7 $\pm$ 4.6	33.6 $\pm$ 4.3	39.6 $\pm$5.9	32.3 $\pm$ 4.0
	Precision	44.0 $\pm$ 8.9	41.9 $\pm$ 10.9	46.5 $\pm$ 9.2	47.5 $\pm$5.9	42.3 $\pm$ 4.9
	F1-score	51.5 $\pm$ 7.3	44.4 $\pm$ 5.6	50.1 $\pm$ 4.7	56.5 $\pm$6.2	48.7 $\pm$ 4.4
Shallow Water	IoU	20.6 $\pm$ 2.8	14.6 $\pm$ 3.0	12.0 $\pm$ 2.0	11.2 $\pm$ 1.4	22.1 $\pm$3.9
	Precision	21.6 $\pm$ 3.0	15.1 $\pm$ 3.2	12.3 $\pm$ 2.0	11.5 $\pm$ 1.4	23.0 $\pm$4.2
	F1-score	34.1 $\pm$ 3.9	25.4 $\pm$ 4.7	21.5 $\pm$ 3.1	20.2 $\pm$ 2.2	36.0 $\pm$5.2
Wakes	IoU	11.8 $\pm$2.6	9.1 $\pm$ 4.6	4.8 $\pm$ 1.5	7.7 $\pm$ 1.2	6.1 $\pm$ 1.2
	Precision	16.4 $\pm$4.6	15.2 $\pm$ 7.7	5.3 $\pm$ 1.9	8.7 $\pm$ 1.4	6.7 $\pm$ 1.5
	F1-score	21.0 $\pm$4.4	16.3 $\pm$ 8.1	9.1 $\pm$ 2.8	14.2 $\pm$ 2.0	11.5 $\pm$ 2.2
Oil Platform	IoU	47.6 $\pm$ 2.1	40.7 $\pm$ 1.4	54.5 $\pm$ 3.1	58.4 $\pm$3.5	49.9 $\pm$ 2.8
	Precision	54.7 $\pm$ 2.4	47.7 $\pm$ 1.8	61.7 $\pm$ 5.5	64.5 $\pm$4.5	54.8 $\pm$ 3.6
	F1-score	64.5 $\pm$ 1.9	57.9 $\pm$ 1.4	70.5 $\pm$ 2.6	73.7 $\pm$2.8	66.5 $\pm$ 2.4
Jellyfish	IoU	4.7 $\pm$ 4.0	0.3 $\pm$ 0.3	5.1 $\pm$ 6.8	4.8 $\pm$ 4.4	10.3 $\pm$14.0
	Precision	31.3 $\pm$ 28.1	2.1 $\pm$ 1.7	21.7 $\pm$ 19.7	85.6 $\pm$10.9	39.6 $\pm$ 24.1
	F1-score	8.7 $\pm$ 7.2	0.7 $\pm$ 0.5	8.9 $\pm$ 11.3	8.9 $\pm$ 7.8	16.1 $\pm$19.8
Clouds	IoU	36.5 $\pm$ 20.3	43.6 $\pm$ 14.3	44.6 $\pm$ 18.8	78.4 $\pm$0.5	47.1 $\pm$ 14.5
	Precision	39.0 $\pm$ 22.9	51.8 $\pm$ 20.6	47.1 $\pm$ 20.5	84.9 $\pm$1.1	48.8 $\pm$ 15.5
	F1-score	50.5 $\pm$ 20.6	59.3 $\pm$ 14.1	59.2 $\pm$ 19.0	87.9 $\pm$0.3	62.8 $\pm$ 13.2
Overall	IoU	29.7 $\pm$ 2.3	27.8 $\pm$ 1.3	32.7 $\pm$ 2.7	41.2 $\pm$1.2	26.8 $\pm$ 0.9
	Precision	40.3 $\pm$ 4.1	36.1 $\pm$ 2.3	42.0 $\pm$ 3.5	56.9 $\pm$1.8	35.4 $\pm$ 2.0
	F1-score	41.6 $\pm$ 2.4	38.5 $\pm$ 1.6	44.6 $\pm$ 2.9	53.3 $\pm$1.3	38.3 $\pm$ 0.8

Table 4.3: Performance comparison of different fusion methods with Bilinear.

4.3 Prediction with the best model

In the last results section, we present the predictions of our best model, **MariNeXt**, evaluating their effectiveness and comparing them with the GTs. This allows us to analyse both the positive and negative aspects of the generated predictions. In particular, some significant examples are shown in Figure 4.1. We decided to divide the results with a black line separating the predictions into two categories: on the left, we find the predictions with good performance, while on the right are those with errors or problems that deserve more in-depth discussion in order to plan possible future improvements. In both parts, we show the RGB images, ground truths and predictions, always following this order.

Analysis of correct predictions (left side) Let us start by analysing the correct predictions. In the first image, we observe that the **clouds** are identified with good accuracy, although not always to their correct extension. The **algae** are also recognised correctly, although slightly denser and coarser than in reality.

The second image shows good identification of **marine debris**. Although these are difficult to distinguish in the RGB image, mainly due to their small size, the model manages to detect them correctly, confirming its ability to operate even on less obvious features.

In the third image, all features are accurately identified, with a particularly good representation of the **land**, which emerges clearly.

The fourth image shows not only the correct identification of the marine debris, but also **turbid water**, confirmed by the RGB image.

In the fifth image, **oil spill** are correctly identified, although not very densely. This suggests that the model is able to recognise the class correctly, but may have difficulty capturing the fine details.

The sixth image represents the **oil platforms**: these are correctly located, but not accurately represented in their shape. Especially for the smaller platforms, the model reconstructs patches that indicate their location, but do not accurately reproduce their structure.

The seventh image shows the presence of **shallow water**, which is correctly identified in all its portions, confirming the goodness of the prediction with respect to the distribution in the RGB image.

Finally, in the eighth image, the model achieves **100%** on all metrics, identifying only **marine water**. However, this gives us a reflection on the usefulness of this prediction: as this is a task aimed at identifying marine pollution phenomena, an image containing only marine water may be of limited use. Furthermore as discussed in the previous chapter, we have applied a class mask such as **marine water**, so in this case the GT will be empty and will not be evaluated.

Analysis of incorrect predictions (right side) Let us now move on to the analysis of incorrect predictions, shown on the right-hand side of Figure 4.1.

In the first image, we only see **marine water**, but the model, confused by the lighter colour compared to the other images, incorrectly classifies the area as **oil spill**. This error does not influence the overall performance negatively, as our mask would exclude the entire scene anyway. However, it is a sign that the model can be improved in discriminating

this class.

The second image highlights **shallow water**, but the model incorrectly classifies the entire clear area as such, ignoring other near features. This could suggest a revision of the masking approach applied during training, as the model appears to correctly recognise areas close to features, but fails on deeper ones. A more extensive mask or an alternative technique could improve this performance.

The third image has an error in identifying **jellyfish**, a class poorly represented in the dataset. This limits the model’s ability to generalise about it, leading it to incorrect predictions. In addition, the presence of **waves** creates further confusion, as they are misclassified as **oil spill**, highlighting a gap in our GTs, where it is complicated to correctly annotate every detail.

The fourth image is particularly interesting: it is an image with a **black border**, a typical problem in the MADOS dataset. The model, failing to interpret this area correctly, partially classifies it as **cloud**, a clearly wrong result. Again, thanks to our mask, there is no deterioration in performance, but the need to improve the treatment of these portions of the image remains evident.

In the fifth image, which is very small and complex, there is some **natural organic material** that the model confuses with **marine debris** and **algae**. Again, the scarcity of examples of this class makes training difficult and suggests the need for more images to improve performance.

The sixth image brings us back to a common mistake: **ships** are often classified as **oil platforms**. This is probably due to their small size and sometimes the similarity between the two classes. Even during the annotation phase, we found it difficult to distinguish ships from platforms, a problem that is reflected in the model’s ability to recognise them correctly.

The seventh image highlights another error concerning the **shallow water**, which is misclassified due to the unexpected clarity of the sea. We also note that the model has difficulty correctly identifying the harbour walkway, classifying it as **oil platform** instead of **land**. This is an expected error, since the walkway does not really belong to the class **land**, but creating a specific class just for it would be impractical.

The last image shows an example of a burnt image, annotated through spectral bands and not through RGB images. The model’s difficulty in correctly classifying this part of the scene, especially near the coast, could be due to an annotation error or a scarcity of useful information in the bands used. The second error is apparently due to the black part that is part of the border of the scene obtained from the satellite.

In general, the results obtained highlight some critical points that need attention: the processing of **minor classes** such as jellyfish, the presence of **black borders** in the images, the accuracy of predictions in **smaller features** such as ships, and the impact of the applied **mask**. These aspects point us in possible future directions to further improve the model, especially in the processing of less represented classes and images with special features.

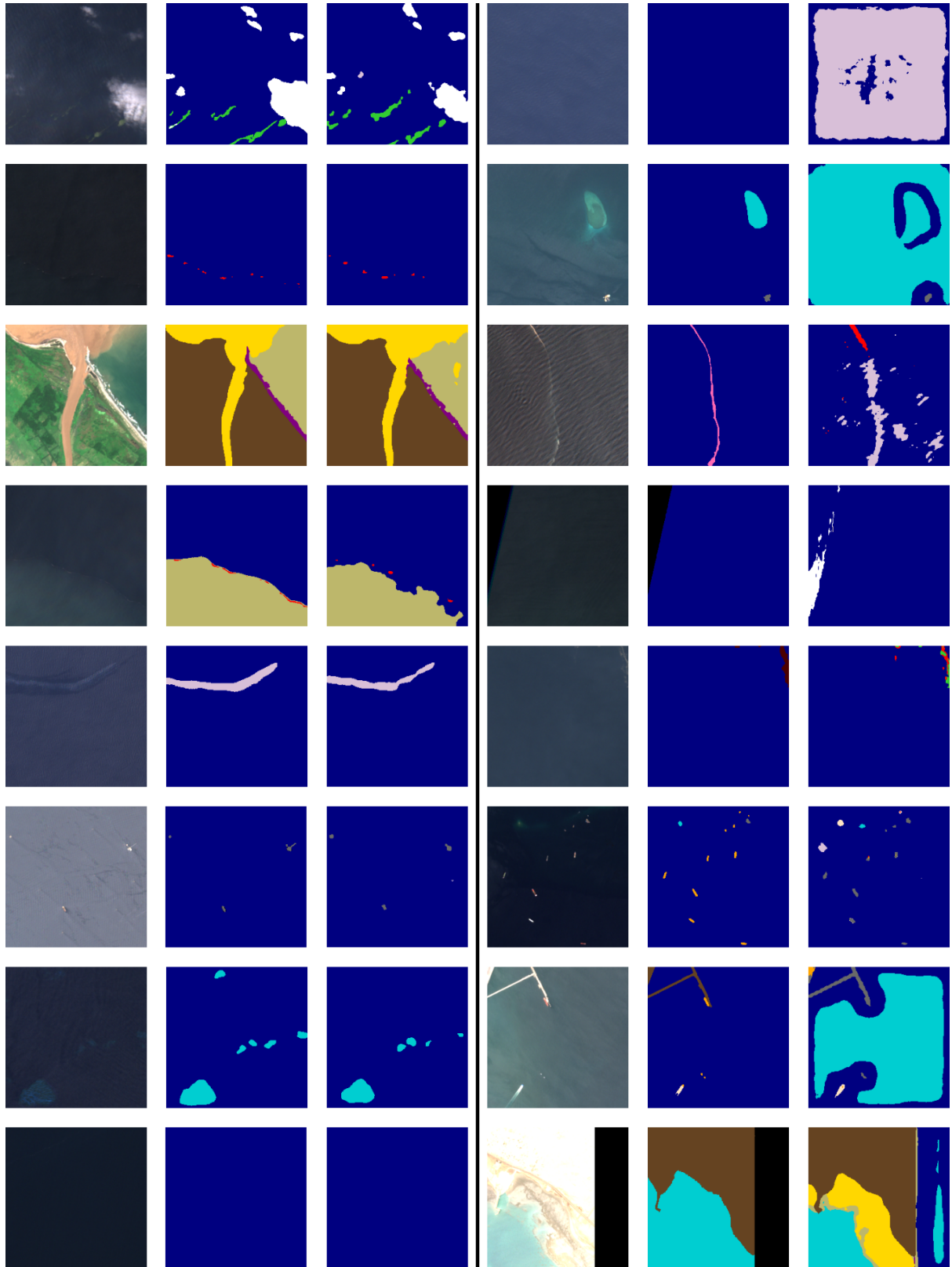


Figure 4.1: *These images show the predictions of the best model, divided into two sections. On the left are the correct predictions, and on the right the incorrect ones, so as to highlight the two types of results. For each case, the PNG image, the GT and the predictions are displayed.*

Chapter 5

Conclusions

This thesis focused primarily on pre-processing data from the MADOS dataset, a global reference collection for detecting marine debris and oil spills from Sentinel-2 satellite images. In particular, we worked on the complete annotation of the GTs, because the previous version, although well done, with good annotations, was poorly annotated and misleading, returning apparently high but false performance. A crucial aspect was the creation of a mask to address the problem of class imbalance, as some classes were over-represented compared to others. This approach allowed us to drastically reduce the majority classes, although, as we noted in the last section on results, this strategy did not always work optimally. For example, in the presence of deeper marine water, the model incorrectly included an unexpected class, as that specific portion had not been considered during the training phase. A further key aspect of the thesis was the development of a model, called **TICINO**, which uses multimodal fusion techniques. We chose this approach because we believe that, when working with multiple bands (i.e. multimodal data), fusion techniques that best capture image features can produce better results than simpler methods, such as joining bands at the beginning and passing them directly to the model (early fusion). As expected, the use of multimodal fusion techniques has indeed led to significant improvements over early fusion. The results obtained with our approach showed a significant increase in metrics, from an initial IoU, precision and F1 score of 29.7%, 40.3% and 41.6%, respectively, to results of 41.2%, 56.9% and 53.3% with **LFV**. This confirms the validity of our hypothesis. However, we were not able to outperform our benchmark model, **MariNeXt**, which performed slightly better, with metrics of 44.0%, 58.9% and 56.8%. However, we are still satisfied with our work, as we have provided a new dataset with complete GTs, effectively addressed the imbalance problem, and confirmed that multimodal fusion techniques are superior to simple early fusion.

Regarding the limitations of the dataset, one of the most critical aspects is the imbalance of the classes. Despite the application of our technique, the dataset remains highly unbalanced, with poorly represented classes such as jellyfish and natural organic material, which limits the overall performance. Another limitation concerns the annotations: because they were done manually, it is possible that errors were made, negatively affecting the training of the model.

In terms of future developments, it would be interesting to obtain new images for the less represented classes in order to reduce the imbalance and improve the accuracy of the model. Another key point concerns upsampling techniques: exploring more advanced methods and improving the quality of images could help the model to better capture information through clearer and more detailed images. In addition, it would be useful to

carry out a review of GTs, involving more experts to guarantee greater accuracy and try new approaches to handle over-represented classes. Finally, one could continue to refine multimodal fusion techniques, in particular **late fusion**, or apply these techniques to the **MariNeXt** model to evaluate if improvements could be achieved there as well, as was the case with the **TICINO** model.

References

- Akbani, Rehan, et al. “Applying support vector machines to imbalanced datasets”. *Machine Learning: ECML 2004: 15th European Conference on Machine Learning, Pisa, Italy, September 20-24, 2004. Proceedings 15*. Springer, 2004, pp. 39–50.
- Andrady, AL, et al. “Marine anthropogenic litter”. *Springer International Publishing, Cham*, 2015, pp. 57–72.
- Azad, Reza, et al. “Medical image segmentation review: The success of u-net”. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.
- Bahdanau, Dzmitry. “Neural machine translation by jointly learning to align and translate”. *arXiv preprint arXiv:1409.0473*, 2014.
- Barbato, Mirko Paolo, et al. “Ticino: A multi-modal remote sensing dataset for semantic segmentation”. *Expert Systems with Applications*, vol. 249, 2024, p. 123600.
- Barbato, Mirko Paolo, et al. “Unsupervised segmentation of hyperspectral remote sensing images with superpixels”. *Remote Sensing Applications: Society and Environment*, vol. 28, 2022, p. 100823.
- Barghout, Lauren. “Visual taxometric approach to image segmentation using fuzzy-spatial taxon cut yields contextually relevant regions”. *Information Processing and Management of Uncertainty in Knowledge-Based Systems: 15th International Conference, IPMU 2014, Montpellier, France, July 15-19, 2014, Proceedings, Part II 15*. Springer, 2014, pp. 163–73.
- Biermann, Lauren, et al. “Finding plastic patches in coastal waters using optical satellite data”. *Scientific reports*, vol. 10, no. 1, 2020, p. 5364.
- Candela, Laura, et al. “The PRISMA mission”. *2016 IEEE international geoscience and remote sensing symposium (IGARSS)*. IEEE, 2016, pp. 253–56.
- Chawla, Nitesh V, et al. “SMOTE: synthetic minority over-sampling technique”. *Journal of artificial intelligence research*, vol. 16, 2002, pp. 321–57.
- Chen, Liang-Chieh, et al. “Rethinking atrous convolution for semantic image segmentation”. *arXiv preprint arXiv:1706.05587*, 2017.
- Chen, Qiang, et al. “Deep domain adaptation for describing people based on fine-grained clothing attributes”. *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2015, pp. 5315–24.
- Ciresan, Dan, et al. “Deep neural networks segment neuronal membranes in electron microscopy images”. *Advances in neural information processing systems*, vol. 25, 2012.
- Contributors, MMSegmentation. MMSegmentation: Openmmlab semantic segmentation toolbox and benchmark. 2020.
- Cordova, Muhammad Reza, and Intan Suci Nurhati. “Major sources and monthly variations in the release of land-derived marine debris from the Greater Jakarta area, Indonesia”. *Scientific Reports*, vol. 9, no. 1, 2019, p. 18730.
- Cordts, Marius, et al. “The cityscapes dataset”. *CVPR Workshop on the Future of Datasets in Vision*. 2015, p. 1.

- Cózar, Andrés, et al. “Marine litter windrows: a strategic target to understand and manage the ocean plastic pollution”. *Frontiers in Marine Science*, vol. 8, 2021, p. 571796.
- Dai, Jifeng, et al. “Deformable convolutional networks”. *Proceedings of the IEEE international conference on computer vision*. 2017, pp. 764–73.
- Dai, Tao, et al. “Second-order attention network for single image super-resolution”. *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2019, pp. 11065–74.
- Diana Earshia, V, and M Sumathi. “A comprehensive study of 1D and 2D image interpolation techniques”. *ICCCE 2018: Proceedings of the International Conference on Communications and Cyber Physical Engineering 2018*. Springer, 2019, pp. 383–91.
- Dierssen, Heidi M. “Hyperspectral measurements, parameterizations, and atmospheric correction of whitecaps and foam from visible to shortwave infrared for ocean color remote sensing”. *Frontiers in Earth Science*, vol. 7, 2019, p. 14.
- Dierssen, Heidi M, and Shungudzemwoyo P Garaba. “Bright oceans: Spectral differentiation of whitecaps, sea ice, plastics, and other flotsam”. *Recent Advances in the Study of Oceanic Whitecaps: Twixt Wind and Waves*, 2020, pp. 197–208.
- Dong, Qi, et al. “Imbalanced deep learning by minority class incremental rectification”. *IEEE transactions on pattern analysis and machine intelligence*, vol. 41, no. 6, 2018, pp. 1367–81.
- Donlon, Craig, et al. “The sentinel-3 mission: Overview and status”. *2012 IEEE International Geoscience and Remote Sensing Symposium*. IEEE, 2012, pp. 1711–14.
- Dosovitskiy, Alexey. “An image is worth 16x16 words: Transformers for image recognition at scale”. *arXiv preprint arXiv:2010.11929*, 2020.
- Drummond, Chris, Robert C Holte, et al. “C4. 5, class imbalance, and cost sensitivity: why under-sampling beats over-sampling”. *Workshop on learning from imbalanced datasets II*. 2003.
- Drusch, Matthias, et al. “Sentinel-2: ESA’s optical high-resolution mission for GMES operational services”. *Remote sensing of Environment*, vol. 120, 2012, pp. 25–36.
- Dvornik, Nikita, et al. “Modeling visual context is key to augmenting object detection datasets”. *Proceedings of the European Conference on Computer Vision (ECCV)*. 2018, pp. 364–80.
- Dwibedi, Debidatta, et al. “Cut, paste and learn: Surprisingly easy synthesis for instance detection”. *Proceedings of the IEEE international conference on computer vision*. 2017, pp. 1301–10.
- Fang, Hao-Shu, et al. “Instaboost: Boosting instance segmentation via probability map guided copy-pasting”. *Proceedings of the IEEE/CVF international conference on computer vision*. 2019, pp. 682–91.
- Feris, Rogerio, et al. “Attribute-based people search: Lessons learnt from a practical surveillance system”. *Proceedings of International Conference on Multimedia Retrieval*. 2014, pp. 153–60.
- Frick, Annett, and Steffen Tervoren. “A framework for the long-term monitoring of urban green volume based on multi-temporal and multi-sensoral remote sensing data”. *Journal of geovisualization and spatial analysis*, vol. 3, no. 1, 2019, p. 6.
- Gao, Shang-Hua, et al. “Res2net: A new multi-scale backbone architecture”. *IEEE transactions on pattern analysis and machine intelligence*, vol. 43, no. 2, 2019, pp. 652–62.
- Garcia-Pineda, Oscar, et al. “Oil spill mapping and measurement in the Gulf of Mexico with textural classifier neural network algorithm (TCNNA)”. *IEEE Journal of Se-*

- lected Topics in Applied Earth Observations and Remote Sensing*, vol. 6, no. 6, 2013, pp. 2517–25.
- Gascon, Ferran, et al. “Copernicus Sentinel-2 mission: products, algorithms and Cal/Val”. *Earth observing systems XIX*. SPIE, 2014, pp. 455–63.
- Geng, Zhengyang, et al. “Is attention better than matrix decomposition?” *arXiv preprint arXiv:2109.04553*, 2021.
- Geyer, Roland, et al. “Production, use, and fate of all plastics ever made”. *Science advances*, vol. 3, no. 7, 2017, e1700782.
- Ghiasi, Golnaz, et al. “Simple copy-paste is a strong data augmentation method for instance segmentation”. *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2021, pp. 2918–28.
- Gonzalez, Rafael C. *Digital image processing*. Pearson education india, 2009.
- Gregor, Karol, et al. “Draw: A recurrent neural network for image generation”. *International conference on machine learning*. PMLR, 2015, pp. 1462–71.
- Guo, Meng-Hao, et al. “Attention mechanisms in computer vision: A survey”. *Computational visual media*, vol. 8, no. 3, 2022, pp. 331–68.
- Guo, Meng-Hao, et al. “Segnext: Rethinking convolutional attention design for semantic segmentation”. *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 1140–56.
- He, Haibo, and Eduardo A Garcia. “Learning from imbalanced data”. *IEEE Transactions on knowledge and data engineering*, vol. 21, no. 9, 2009, pp. 1263–84.
- He, Kaiming, et al. “Deep residual learning for image recognition”. *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 770–78.
- He, Qibin, et al. “Multimodal remote sensing image segmentation with intuition-inspired hypergraph modeling”. *IEEE Transactions on Image Processing*, vol. 32, 2023, pp. 1474–87.
- Hedjazi, Mohamed Abbas, et al. “On identifying leaves: A comparison of CNN with classical ML methods”. *2017 25th signal processing and communications applications conference (SIU)*. IEEE, 2017, pp. 1–4.
- Hochreiter, Sepp, et al. Gradient flow in recurrent nets: the difficulty of learning long-term dependencies. 2001.
- Hong, Danfeng, et al. “More diverse means better: Multimodal deep learning meets remote-sensing imagery classification”. *IEEE Transactions on Geoscience and Remote Sensing*, vol. 59, no. 5, 2020, pp. 4340–54.
- Hospedales, Timothy M, et al. “Finding rare classes: Active learning with generative and discriminative models”. *IEEE transactions on knowledge and data engineering*, vol. 25, no. 2, 2011, pp. 374–86.
- Hou, Qibin, et al. “Strip pooling: Rethinking spatial pooling for scene parsing”. *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2020, pp. 4003–12.
- Hu, Chuanmin. “Remote detection of marine debris using satellite observations in the visible and near infrared spectral range: Challenges and potentials”. *Remote Sensing of Environment*, vol. 259, 2021, p. 112414.
- Hu, Chuanmin, et al. “Spectral and spatial requirements of remote measurements of pelagic Sargassum macroalgae”. *Remote Sensing of Environment*, vol. 167, 2015, pp. 229–46.

- Hu, Chuanmin, et al. "Spectral characteristics of sea snout reflectance observed from satellites: Implications for remote sensing of marine debris". *Remote Sensing of Environment*, vol. 269, 2022, p. 112842.
- Hu, Jie, et al. "Squeeze-and-excitation networks". *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018, pp. 7132–41.
- Ioffe, Sergey. "Batch normalization: Accelerating deep network training by reducing internal covariate shift". *arXiv preprint arXiv:1502.03167*, 2015.
- Jambeck, Jenna R, et al. "Plastic waste inputs from land into the ocean". *science*, vol. 347, no. 6223, 2015, pp. 768–71.
- Jang, Yong Chang, et al. "Estimation of the annual flow and stock of marine debris in South Korea for management purposes". *Marine pollution bulletin*, vol. 86, nos. 1–2, 2014, pp. 505–11.
- Japkowicz, Nathalie, and Shaju Stephen. "The class imbalance problem: A systematic study". *Intelligent data analysis*, vol. 6, no. 5, 2002, pp. 429–49.
- Jiang, Xingyu, et al. "A review of multimodal image matching: Methods and applications". *Information Fusion*, vol. 73, 2021, pp. 22–71.
- Kaiming, He, et al. "Mask r-cnn". *Proceedings of the IEEE international conference on computer vision*. 2017, pp. 2961–69.
- Kanjir, Urška, et al. "Vessel detection and classification from spaceborne optical images: A literature survey". *Remote sensing of environment*, vol. 207, 2018, pp. 1–26.
- Kikaki, Aikaterini, et al. "Remotely sensing the source and transport of marine plastic debris in Bay Islands of Honduras (Caribbean Sea)". *Remote Sensing*, vol. 12, no. 11, 2020, p. 1727.
- Kikaki, Katerina, et al. "Detecting Marine pollutants and Sea Surface features with Deep learning in Sentinel-2 imagery". *ISPRS Journal of Photogrammetry and Remote Sensing*, vol. 210, 2024, pp. 39–54.
- Kikaki, Katerina, et al. "MARIDA: A benchmark for Marine Debris detection from Sentinel-2 remote sensing data". *PloS one*, vol. 17, no. 1, 2022, e0262247.
- Kimura, Masanari. "Understanding test-time augmentation". *International Conference on Neural Information Processing*. Springer, 2021, pp. 558–69.
- Kirillov, Alexander, et al. "Panoptic segmentation". *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2019, pp. 9404–13.
- Klemas, Victor. "Airborne remote sensing of coastal features and processes: An overview". *Journal of Coastal Research*, vol. 29, no. 2, 2013, pp. 239–55.
- Krawczyk, Bartosz. "Learning from imbalanced data: open challenges and future directions". *Progress in artificial intelligence*, vol. 5, no. 4, 2016, pp. 221–32.
- Kress, Gunther. "Multimodal discourse: The modes and media of contemporary communication". *Edward Arnold*, 2001.
- Kubryakov, AA, et al. "Application of Landsat imagery for the investigation of wave breaking". *Remote Sensing of Environment*, vol. 253, 2021, p. 112144.
- Lebreton, Laurent CM, et al. "River plastic emissions to the world's oceans". *Nature communications*, vol. 8, no. 1, 2017, p. 15611.
- Leifer, Ira, et al. "State of the art satellite and airborne marine oil spill remote sensing: Application to the BP Deepwater Horizon oil spill". *Remote Sensing of Environment*, vol. 124, 2012, pp. 185–209.
- Li, Jiaxin, et al. "Deep learning in multimodal remote sensing data fusion: A comprehensive review". *International Journal of Applied Earth Observation and Geoinformation*, vol. 112, 2022, p. 102926.

- Li, Yabei, et al. “Semantics-guided multi-level RGB-D feature fusion for indoor semantic segmentation”. *2017 IEEE international conference on image processing (ICIP)*. IEEE, 2017, pp. 1262–66.
- Liu, Yingfei, et al. “A novel technique for ship wake detection from optical images”. *Remote Sensing of Environment*, vol. 258, 2021, p. 112375.
- Lloyd, Annemaree, and Sanna Talja. *Practising information literacy: Bringing theories of learning, practice and information literacy together*. Elsevier, 2010.
- Long, Jonathan, et al. “Fully convolutional networks for semantic segmentation”. *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2015, pp. 3431–40.
- Louis, Jérôme, et al. “Sentinel-2 Sen2Cor: L2A processor for users”. *Proceedings living planet symposium 2016*. Spacebooks Online, 2016, pp. 1–8.
- Luong, Minh-Thang. “Effective approaches to attention-based neural machine translation”. *arXiv preprint arXiv:1508.04025*, 2015.
- Maciejewski, Tomasz, and Jerzy Stefanowski. “Local neighbourhood extension of SMOTE for mining imbalanced data”. *2011 IEEE symposium on computational intelligence and data mining (CIDM)*. IEEE, 2011, pp. 104–11.
- Martínez-Vicente, Víctor, et al. “Measuring marine plastic debris from space: Initial assessment of observation requirements”. *Remote Sensing*, vol. 11, no. 20, 2019, p. 2443.
- Morelli, Veronica Grazia, et al. “Multimodal Fusion Methods with Vision Transformers for Remote Sensing Semantic Segmentation”. *2023 13th Workshop on Hyperspectral Imaging and Signal Processing: Evolution in Remote Sensing (WHISPERS)*. IEEE, 2023, pp. 1–5.
- O’Mahony, Niall, et al. “Deep learning vs. traditional computer vision”. *Advances in Computer Vision: Proceedings of the 2019 Computer Vision Conference (CVC), Volume 1 1*. Springer, 2020, pp. 128–44.
- Peng, Chao, et al. “Large kernel matters—improve semantic segmentation by global convolutional network”. *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2017, pp. 4353–61.
- Qi, Lin, et al. “In search of floating algae and other organisms in global oceans and lakes”. *Remote Sensing of Environment*, vol. 239, 2020, p. 111659.
- Qin, Zhihao, et al. “A methodology for true orthorectification of large-scale urban aerial images and automatic detection of building occlusions using digital surface model”. *IGARSS 2003. 2003 IEEE International Geoscience and Remote Sensing Symposium. Proceedings (IEEE Cat. No. 03CH37477)*. IEEE, 2003, pp. 729–31.
- Ramachandram, Dhanesh, and Graham W Taylor. “Deep multimodal learning: A survey on recent advances and trends”. *IEEE signal processing magazine*, vol. 34, no. 6, 2017, pp. 96–108.
- Ronneberger, Olaf, et al. “U-net: Convolutional networks for biomedical image segmentation”. *Medical image computing and computer-assisted intervention—MICCAI 2015: 18th international conference, Munich, Germany, October 5-9, 2015, proceedings, part III 18*. Springer, 2015, pp. 234–41.
- Salcedo-Sanz, Sancho, et al. “Machine learning information fusion in Earth observation: A comprehensive review of methods, applications and data sources”. *Information Fusion*, vol. 63, 2020, pp. 256–72.
- Schmidt, Christian, et al. “Export of plastic debris by rivers into the sea”. *Environmental science & technology*, vol. 51, no. 21, 2017, pp. 12246–53.

- Sishodia, Rajendra P, et al. "Applications of remote sensing in precision agriculture: A review". *Remote sensing*, vol. 12, no. 19, 2020, p. 3136.
- Srivastava, Nitish, et al. "Dropout: a simple way to prevent neural networks from over-fitting". *The journal of machine learning research*, vol. 15, no. 1, 2014, pp. 1929–58.
- Sun, Shaojie, et al. "Remote sensing assessment of oil spills near a damaged platform in the Gulf of Mexico". *Marine Pollution Bulletin*, vol. 136, 2018, pp. 141–51.
- Szegedy, Christian, et al. "Going deeper with convolutions". *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2015, pp. 1–9.
- Tang, Yuchun, et al. "SVMs modeling for highly imbalanced classification". *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, vol. 39, no. 1, 2008, pp. 281–88.
- Ting, Kai Ming. "A comparative study of cost-sensitive boosting algorithms". *Proceedings of the seventeenth international conference on machine learning*. 2000, pp. 983–90.
- Topouzelis, Konstantinos, et al. "Detection of floating plastics from satellite and unmanned aerial systems (Plastic Litter Project 2018)". *International Journal of Applied Earth Observation and Geoinformation*, vol. 79, 2019, pp. 175–83.
- Van der Zande, Dimitry, et al. "JellySpec: Feasibility study for determining the spectral characteristics of jellyfish from Belgian waters". *Proceedings of the Ocean Optics XXII Conference, Portland, OR, USA*. 2014, pp. 26–31.
- Van Emmerik, Tim, et al. "Manila river mouths act as temporary sinks for macroplastic pollution". *Frontiers in Marine Science*, vol. 7, 2020, p. 545812.
- Van Westen, CJ. "Remote sensing for natural disaster management". *International archives of photogrammetry and remote sensing*, vol. 33, B7/4, PART 7, 2000, pp. 1609–17.
- Vanhellemont, Quinten, and Kevin Ruddick. "Atmospheric correction of metre-scale optical satellite data for inland and coastal water applications". *Remote sensing of environment*, vol. 216, 2018, pp. 586–97.
- Vaswani, A. "Attention is all you need". *Advances in Neural Information Processing Systems*, 2017.
- Wang, Jingdong, et al. "Deep high-resolution representation learning for visual recognition". *IEEE transactions on pattern analysis and machine intelligence*, vol. 43, no. 10, 2020, pp. 3349–64.
- Weiss, Gary M. "Mining with rarity: a unifying framework". *ACM Sigkdd Explorations Newsletter*, vol. 6, no. 1, 2004, pp. 7–19.
- Wellmann, Thilo, et al. "Remote sensing in urban planning: Contributions towards ecologically sound policies?" *Landscape and urban planning*, vol. 204, 2020, p. 103921.
- Wightman, Ross, et al. Pytorch image models. 2019.
- Xie, Enze, et al. "SegFormer: Simple and efficient design for semantic segmentation with transformers". *Advances in neural information processing systems*, vol. 34, 2021, pp. 12077–90.
- Yu, Fisher, and Vladlen Koltun. "Multi-scale context aggregation by dilated convolutions". *arXiv preprint arXiv:1511.07122*, 2015.
- Zhang, Ning, et al. "Panda: Pose aligned networks for deep attribute modeling". *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2014, pp. 1637–44.
- Zhao, Hengshuang, et al. "Pyramid scene parsing network". *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2017, pp. 2881–90.

- Zhao, Jun, et al. “Exploring the potential of optical remote sensing for oil spill detection in shallow coastal waters-a case study in the Arabian Gulf”. *Optics Express*, vol. 22, no. 11, 2014, pp. 13755–72.
- Zhao, Shiye, et al. “Analysis of suspended microplastics in the Changjiang Estuary: Implications for riverine plastic load to the ocean”. *Water research*, vol. 161, 2019, pp. 560–69.

Acknowledgements

We have come to the conclusion of this thesis. In this section, I would like to apologize to non-Italian readers, as the people I will mention are Italian. Therefore, I will write this part in my native language so that everyone can understand.

Per iniziare questi ringraziamenti, desidero esprimere la mia gratitudine alle persone più importanti per lo sviluppo di questo progetto, in particolare al professor **Paolo Napoletano** e a **Mirko Paolo Barbato**. Ringrazio il mio relatore per avermi coinvolto in questo straordinario progetto, permettendomi di collaborare nel suo laboratorio. Grazie a lui, ho avuto l'opportunità di imparare molte cose che non avevo mai visto prima. Inoltre, lo ringrazio per la sua grande disponibilità nei miei confronti e per la sua gentilezza, dimostrata costantemente da marzo fino ad oggi.

Un ringraziamento speciale va anche a Mirko, per il suo supporto tecnico. In diverse occasioni, quando mi trovavo di fronte a difficoltà dovute alle mie limitate conoscenze, come l'uso di PyTorch o delle macchine virtuali, Mirko è stato sempre pronto ad aiutarmi. Grazie di cuore anche a te per la tua immensa disponibilità. A entrambi, il mio più sincero grazie. Non avrei mai immaginato di concludere il mio percorso universitario affiancato da persone come voi. Grazie per aver aumentato la mia passione in questo campo e per avermi dato la possibilità di apprendere meglio aspetti come la segmentazione semantica, l'utilizzo di PyTorch e la scrittura futura, di un articolo scientifico condiviso.

Dopo questi ringraziamenti iniziali per la tesi, vorrei rivolgere un pensiero alle persone più importanti, quelle che sono state presenti fin dal primo giorno. Questo ringraziamento non è solo per il percorso della laurea magistrale, ma per tutto il cammino intrapreso dalle elementari fino ad oggi, poiché si conclude una parte importante della mia vita.

In primo luogo, desidero ringraziare i miei genitori. Non so da dove cominciare: ci sarebbero tante cose da dire, e forse non sono bravo in questo, ma posso dirvi con certezza grazie per esserci sempre stati, per avermi sostenuto e cresciuto. Grazie perché oggi sono qui, con questa doppia laurea: questi piccoli passi, uno dopo l'altro, sono il risultato del vostro impegno e del vostro amore. Senza di voi, tutto questo non sarebbe stato possibile.

Un ringraziamento speciale a mia **Mamma**: la tua forza, disponibilità, il tuo sostegno e il tuo amore incondizionato sono stati la mia forza durante tutto il percorso. Mi hai insegnato a credere in me stesso e a non arrendermi, anche nei momenti più difficili.

Un ringraziamento speciale va anche a mio **Papà**: la tua fiducia in me e il tuo incoraggiamento sono stati una fonte di forza lungo questo viaggio. Ogni passo verso questo traguardo è stato possibile grazie ai momenti che mi hai dedicato per studiare insieme e all'educazione che mi hai trasmesso. Sono grato per le tue parole di saggezza e per il tuo sostegno nei momenti di difficoltà.

Vorrei fare un ringraziamento speciale alle altre quattro persone che sono state presenti fin dal primo giorno. A mio zio **Rocco** e mia zia **Gianna**: un grazie di cuore per avermi sempre sostenuto e incoraggiato. La vostra presenza nella mia vita è stata fondamentale per permettermi di raggiungere questo traguardo. Vi voglio bene.

E un ringraziamento va anche a **Giuseppe** e **Giulio**, i miei cugini maggiori. Anche se anni fa mi trattavate come il più piccolo, facendomi scherzi e dispetti, ci siamo sempre voluti bene. Come tutti quelli che ci sono stati fin dal giorno uno, siete sempre stati presenti e disponibili nella mia vita, condividendo momenti di scherzi, divertimento e scelte importanti che ho sempre tenuto in considerazione. Grazie per far parte della mia famiglia e per essere qui con me oggi.

Inoltre, vorrei aprire una parentesi per ringraziare anche **Valentina** e **Miriam**: anche se non siete con noi fin dal giorno uno, ormai fate parte di questa famiglia. Grazie di cuore.

Passiamo ora ai miei compagni universitari, che sono stati presenti soprattutto in questi ultimi due anni. Come prima persona, vorrei ringraziare **Valentina**: abbiamo passato il primo anno di università sempre insieme, trascorrendo anche delle volte giornate intere, dalle 9 alle 18, supportandoci a vicenda, scherzando e ascoltando le bellissime lezioni insieme ogni giorno. Mi dispiace se il secondo anno ci siamo visti di meno, sia perché sono andato in Germania, sia per via della tesi e dei pochi corsi che seguivo nel secondo semestre. Tuttavia, sei stata sicuramente la compagna di corso più speciale, e ti voglio davvero bene. Grazie per aver condiviso con me questa magistrale.

Voglio inoltre ringraziare tutti i compagni che ho conosciuto, in particolare **Luca** e **Fabio**. Oltre a semplici compagni di corso, abbiamo lavorato insieme e condiviso diversi spritz post-esame. Grazie anche a voi!

Infine, un ringraziamento speciale va a tutte le persone che ho conosciuto nella mia vita, dal giorno uno fino ad oggi: amici e conoscenti, persone incontrate per un'ora o da anni, persone simpatiche e altre da evitare, persone scherzose, interessanti, noiose, timide, colte, logorroiche e molte altre ancora. A tutti voi dico grazie, perché se oggi sono la persona che sono, il merito, anche se solo in parte, è anche vostro.

Grazie di cuore a tutti per aver reso possibile questo traguardo e per aver contribuito a formare la persona che sono oggi. Concludo questo percorso con la consapevolezza di non essere mai stato solo e con l'entusiasmo di affrontare le sfide future con tutto quello che mi avete insegnato.

Qui si concludono i miei ringraziamenti, ma voglio dedicare un ultimo pensiero a una persona speciale che non voglio mettere insieme agli altri. Non sarà importante quanto chi è stato con me fin dal primo giorno, ma in questi due anni è stata la persona più incisiva nella mia vita, e per questo voglio esprimere al meglio ciò che ho vissuto.

Vorrei ringraziarti, a te che hai iniziato questa avventura con me, passando molti momenti insieme. Non so ancora come definirti: compagno, amico, fratello, socio, collega, compagno di bevute post-esame o addirittura coinquilino. Vorrei anche fare una parentesi per ringraziare Knime per averci permesso di vivere un'esperienza fantastica insieme, ospitandoci e offrendo questa challenge che ci ha dato un'opportunità indimenticabile. Tornando a noi, non so da dove iniziare a ringraziarti. Ci sono così tante cose. Sicuramente, se oggi sono qui e il voto che otterrò è anche merito tuo. In molti esami e progetti, grazie a te sono riuscito a ottenere voti alti e a consegnarli in tempo. Vorrei menzionare esami come *Visual Information Processing and Management* e *Streaming Data Management and Time Series Analysis*: senza di te, sarei ancora lì a provarli. Inoltre, voglio ringraziarti per la challenge. La nostra vittoria è stata possibile grazie a te: senza quei 20 punti del paper, probabilmente non ci saremmo classificati tra i primi cinque. Ti sono grato per le ore trascorse insieme a studiare e progettare, anche se molti dei nostri lavori non sono stati apprezzati, molti erano bellissimi (come la dashboard degli Airbnb di Milano). Mi dispiace che oggi non sei qui con me per concludere insieme questo ciclo

che abbiamo iniziato. Ricorderò per sempre i bei momenti, gli aperitivi, le serate passate insieme, i tre mesi in Germania, le passeggiate sul lago, le uscite con i colleghi, gli sport with Tina, i deliziosi crauti, le serate con Assassination Classroom, i momenti con i nostri nuovi coinquilini (chi più e chi meno), le interminabili notti sui maledetti progetti in matlab e le nostre idee di vivere lì, fino a non voler più tornare in Germania. Ti ringrazio anche per avermi aperto gli occhi su molte cose, sia dal punto di vista personale sia su cosa volere dalla vita. Grazie a te e alle nostre lunghe chiacchierate, ora ho un'idea più chiara di cosa mi piace. Non so ancora se seguirò questa strada, ma avere un'idea è sicuramente meglio. Più scrivo, più mi vengono in mente ricordi, come il nostro paper per la conferenza ECML. Ho perso il conto delle cose fatte insieme in questi due anni e non so più quante volte ti devo ringraziare, ma sicuramente tutte sono state fantastiche, come scrivere quel paper per l'esame di Social. Non so ancora come ci sia riuscito, ma è stato bello (credo). Penso di fermarmi qui. Ti ho dato abbastanza notorietà, sei stato una persona speciale in questi anni, saccente, ma speciale. L'unica cosa che mi resta da dire ora, dopo tutto ciò che abbiamo condiviso, dopo i momenti belli e le sfide affrontate insieme, è...

Grazie **Vittorio**, grazie per tutto.